

DEGREE PROJECT FOR MASTER OF SCIENCE WITH SPECIALIZATION IN ROBOTICS
DEPARTMENT OF ENGINEERING SCIENCE
UNIVERSITY WEST

Conversational Task Configuration for Modular Pick-and-Place Robotics

Resolving Instruction Ambiguity
through Vision-Grounded Dialogue

Sandro Gabl



A THESIS SUBMITTED TO THE DEPARTMENT OF ENGINEERING SCIENCE
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE WITH SPECIALIZATION IN ROBOTICS
AT UNIVERSITY WEST

2026

Date:	June 2, 2026
Author:	Sandro Gabl
Examiner:	Prof. Mattias Bennulf
Advisor:	Xiaoxiao Zhang
Program:	Master Program in Robotics and Automation
Main field of study:	Automation with a specialization in industrial robotics
Credits:	120 Higher Education credits
Keywords:	conversational task configuration, vision-guided robotics, large language models, PDDL planning, high-mix low-volume manufacturing
Publisher:	University West, Department of Engineering Science S-461 86 Trollhättan, SWEDEN Phone: + 46 520 22 30 00 Fax: + 46 520 22 32 99 Web: www.hv.se

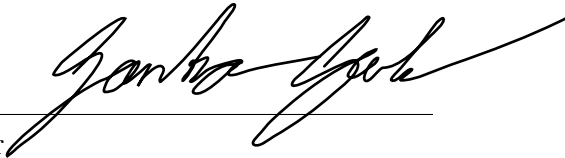
Abstract

High-mix, low-volume manufacturing requires robotic systems to be reconfigured frequently, yet the specialist knowledge required for such reconfiguration limits flexibility on the shop floor. This thesis investigates whether non-expert operators can perform semantic, task-level reconfiguration of a collaborative robotic pick-and-place system through natural language dialogue grounded in vision-based perception. An integrated framework was designed and implemented, comprising a vision module for object detection and localisation, a multi-turn LLM dialogue module with ambiguity detection and constraint checking, a configuration module that maintains a symbolic world state across sessions, a PDDL planner that produces verified motion sequences, and an execution module that drives a Franka Emika Panda robot. The system was evaluated across four scenarios of increasing complexity, from single-attribute baseline configuration to a full end-to-end production changeover, with deliberately ambiguous instruction phrasing and constraint-violating inputs. Across 45 configuration sessions the system produced correct configurations in all cases, 40 of them without operator intervention beyond answering the system’s own clarifications. In a separate set of 14 constraint violation sub-trials, 12 violations were detected. Referential ambiguity was reliably resolved with a fixed two-turn overhead through scene-grounded clarification, whereas omission ambiguity was the dominant cost driver in domains with many interdependent parameters. Constraint violations rooted in factual contradictions against the workspace state were caught reliably, but cross-attribute logical conflicts were not. The findings demonstrate that conversational, vision-guided task configuration is feasible as a reconfiguration mechanism for modular pick-and-place systems, and identify the structural drivers of dialogue cost (in particular, the absence of proactive parameter completeness checking and the stateless constraint-violation handler) that any comparative efficiency assessment against alternative reconfiguration approaches would need to account for.

Affirmation

This master degree report, *Conversational Task Configuration for Modular Pick-and-Place Robotics*, was written as part of the master degree work needed to obtain a Master of Science with specialisation in Robotics degree at University West. All material in this report, that is not my own, is clearly identified and used in an appropriate and correct way. The main part of the work included in this degree project has not previously been published or used for obtaining another degree.

Signature by the author



01.06.2026

Date

Sandro Gabl

Contents

Abstract	ii
Affirmation	iii
List of Figures and Tables	vii
Symbols and Glossary	viii
1 Introduction	1
1.1 Industrial context and motivation	1
1.2 Problem description	2
1.3 Aim and objectives	3
1.4 Research questions	4
1.5 Scope and limitations	4
1.6 Thesis structure	5
2 Related work	6
2.1 Robot reconfiguration and task-level programming	6
2.2 Modular robotic manipulation systems	7
2.3 Human-centred robot programming and end-user programming	8
2.4 Dialogue-based and language-based robot control	9
2.5 Clarification, ambiguity and mixed-initiative interaction	10
2.6 Vision-based grounding and perception for task specification	12
2.7 Classical task planning and automated sequence generation	13
2.8 Summary of related research and identified research gaps	15
3 Method	17
3.1 Research methodology	17
3.2 Overall system architecture	18
3.3 High-level configuration attributes	22
3.4 Experimental design	23
3.4.1 Per-type ambiguity evaluation	23
3.4.2 Ambiguity type definitions	24
3.4.3 Turn type definitions	25
3.4.4 Results presentation	25
3.4.5 Constraint violation handling	26
3.5 Sources of error and uncertainty	26
3.6 Experiment evaluation	26
4 Design and Implementation	27
4.1 Hardware platform	27
4.2 Physical workspace design and fabrication	27
4.2.1 Parts	28
4.2.2 Kits and containers	28
4.3 Vision module	30
4.3.1 Camera configuration and image acquisition	30
4.3.2 Metric workspace calibration	30

Contents

4.3.3	Container and kit localisation	31
4.3.4	Part detection pipeline	31
4.3.5	Data structuring and output	32
4.3.6	Module components	33
4.4	Communication module	34
4.4.1	System prompt engineering	34
4.4.2	Conversation flow management	35
4.4.3	Ambiguity detection system	35
4.4.4	Tool-augmented conversation	35
4.4.5	Block parsing and validation	36
4.4.6	User intent classification	36
4.4.7	Conflict detection and resolution	36
4.4.8	Output structuring	37
4.4.9	Update scene dialogue	37
4.4.10	Cancellation mechanism	37
4.4.11	Module components	38
4.5	Configuration module	39
4.5.1	State structure	39
4.5.2	Applying configuration changes	39
4.5.3	Scene update pipeline	40
4.5.4	Post-execution state update	40
4.5.5	Module components	41
4.6	Planner module (PDDL planner)	42
4.6.1	Planner selection and priority enforcement	42
4.6.2	PDDL domain architecture	42
4.6.3	Problem generation	43
4.6.4	Fast Downward integration	43
4.6.5	Plan parsing and sequence interface	43
4.6.6	Module components	44
4.7	Execution module	45
4.7.1	Robot control foundation	45
4.7.2	Pick-and-place cycle	45
4.7.3	Sequence execution	45
4.7.4	Position handling	46
4.7.5	Error handling and recovery	46
4.7.6	Coupling to other modules	46
4.7.7	Module components	46
4.8	User interface	47
4.8.1	GUI architecture	47
4.8.2	Session management	47
4.8.3	Module components	47
4.9	System infrastructure	48
4.9.1	Module components	48
5	Results and Discussion	49
5.1	Scenario 1 — Baseline attribute configuration	49

Contents

5.1.1	Results	50
5.1.2	Discussion	51
5.2	Scenario 2 — Update workspace configuration	52
5.2.1	Results	52
5.2.2	Discussion	53
5.3	Scenario 3, Experiment 1 — Task specification reconfiguration (sorting)	55
5.3.1	Results	56
5.3.2	Discussion	56
5.4	Scenario 3, Experiment 2 — Task specification reconfiguration (kitting)	58
5.4.1	Results	59
5.4.2	Discussion	59
5.5	Scenario 4 — Full end-to-end reconfiguration cycle	61
5.5.1	Results	62
5.5.2	Discussion	62
5.6	Cross-scenario analysis	64
5.6.1	RQ1: configuration correctness across complexity	64
5.6.2	RQ2: ambiguity detection and resolution	65
5.6.3	RQ3: structural drivers of dialogue cost	67
5.6.4	Overall system assessment	69
6	Conclusion	71
6.1	Critical discussion	72
6.2	Generalisation of the result	74
6.2.1	Applicability to other domains	74
6.2.2	Uniqueness and position in the literature	75
6.3	Future work and research	76
7	References	78

List of Figures and Tables

Figures

1	Operational session flow of the proposed framework.	19
2	Overall system architecture of the proposed framework.	21
3	3D-printed part variations	28
4	3D-printed container (left) and kit (right).	28
5	The workspace used for experimental pick-and-place operations . . .	29
6	Experimental setup scenario 1.	49
7	Experimental setup scenario 2.	52
8	Experimental setup scenario 3 experiment 1.	55
9	Experimental setup scenario 3 experiment 2.	58
10	Experimental setup scenario 4.	62

Tables

1	High-level configuration attributes supported by the system.	22
2	Vision module scripts.	33
3	Communication module scripts.	38
4	Configuration module scripts.	41
5	Planning module scripts.	44
6	Execution module scripts.	46
7	User interface scripts.	47
8	System infrastructure scripts.	48
9	Scenario 1 — Baseline reconfiguration results.	50
10	Scenario 2 — Workspace update ambiguity results.	53
11	Scenario 2 — Constraint violation results.	53
12	Scenario 3, Experiment 1 — Sorting task ambiguity results.	56
13	Scenario 3, Experiment 1 — Sorting task constraint violation results. .	56
14	Scenario 3, Experiment 2 — Kitting task ambiguity results.	59
15	Scenario 3, Experiment 2 — Kitting task constraint violation results. .	59
16	Scenario 4 — Full end-to-end cycle results by phase.	62
17	Cross-scenario turn-type summary (all trials including CV sub-trials). .	64

Symbols and Glossary

API	Application Programming Interface
AST-CLAR	Assistant clarification turn (turn-type label)
AST-PROP	Assistant proposal turn (turn-type label)
AST-VIOL	Assistant constraint-violation turn (turn-type label)
A*	A heuristic graph search algorithm used by classical planners
ChArUco	Combined chessboard and ArUco fiducial pattern used for camera calibration
CLAHE	Contrast-Limited Adaptive Histogram Equalisation
CV	Constraint violation (a sub-trial type used in the evaluation)
FCI	Franka Control Interface
FF heuristic	Fast-Forward heuristic used in classical planning search
GUI	Graphical User Interface
HMLV	High-mix, low-volume (production paradigm)
HSV	Hue, Saturation, Value colour space
I	Implicit constraint ambiguity (operational ambiguity type)
LLM	Large Language Model
MoveIt	Motion planning framework for ROS
O	Omission ambiguity (operational ambiguity type)
OMPL	Open Motion Planning Library
PDDL	Planning Domain Definition Language
PLC	Programmable Logic Controller
R	Referential ambiguity (operational ambiguity type)
ROI	Region of Interest
ROS	Robot Operating System
S	Spatial ambiguity (operational ambiguity type)
SME	Small and Medium-sized Enterprise
USR-CLAR	User clarification-response turn (turn-type label)
USR-CONF	User confirmation turn (turn-type label)
USR-INST	User instruction turn (turn-type label)

1. Introduction

This chapter establishes the industrial and scientific context motivating the thesis. It outlines the problem being addressed, the aim and objectives pursued, the research questions guiding the investigation, and the scope within which the work is conducted.

1.1 Industrial context and motivation

Industrial manufacturing was long dominated by mass production, in which dedicated lines, specialised tooling, and rigid process sequences maximised throughput at low unit cost, amortising large upfront investment over high volumes of a stable, long-lived product. Globalisation, competition, and demand for personalisation have since shifted production toward differentiation: manufacturers now offer many variants within a product family, and even minor individual changes combine into non-standard routings, varying process times, and frequent setup changes, while shortened product life cycles demand rapid ramp-up and phase-out [1, 2]. Static, dedicated lines accommodate such change poorly, as the mechanical, tooling, and software reconfiguration involved interrupts production and drives reduced utilisation, downtime, and higher cost [1]. Many industries have therefore moved toward high-mix, low-volume manufacturing of made-to-order or engineered-to-order products, motivating the search for flexible, reconfigurable solutions. Robotic automation is often seen as a key enabler here, since industrial robots are multipurpose manipulators whose behaviour can in principle be adapted through software rather than physical modification, allowing one platform to serve material handling, assembly, or inspection. In practice this flexibility is constrained by how robotic systems are engineered and programmed: static cell layouts, tightly coupled tooling, and offline workflows assume stable task definitions, so reconfiguring behaviour requires modifying programs, adjusting layouts, and re-validating safety constraints, a process that is time-consuming and dependent on expert programmers [3, 4]. This dependence prevents shop-floor personnel from adapting robots directly, so manufacturers often retain manual processes for frequently changing tasks, reducing the economic viability of conventional robotic automation in high-mix environments. Collaborative robots have gained attention in this setting, since they operate safely near humans and offer more intuitive programming than traditional industrial robots. Kinesthetic teaching, walk-through programming, and programming by demonstration let non-experts specify motions by physically guiding the robot or demonstrating actions, lowering the programming barrier [5, 6]. However, teaching-based methods still require manual effort per modification, and as task complexity and variability grow, the cumulative effort of repeated teaching becomes substantial [6]. Reconfiguration in collaborative systems is also typically limited to parameter changes rather than structural adaptation [7], and efficient task-level reconfiguration remains an open challenge.

1.2 Problem description

In high-mix, low-volume environments, frequent changes in tasks, variants, and workspace layouts require robotic systems to be reconfigured quickly and reliably. Slow or error-prone reconfiguration causes downtime, reduced utilisation, and higher cost, while increasing the workload of operators who must repeatedly intervene under time pressure [1, 4].

A central reason reconfiguration effort remains high is a mismatch in level of description. Operators describe task changes informally, in terms of goals, object roles, and priorities, yet translating this intent into concrete robot configurations still requires manual parameter tuning, repeated teaching, and validation by experts [3, 5, 6]. The dominant shop-floor tools (recipe files, PLC configuration screens, teach pendants) operate at the level of motion primitives and named waypoints and offer no mechanism for expressing task semantics. Changes an operator could state in one sentence, such as “the red parts are fragile” or “prioritise blue over red”, therefore cannot be enacted without specialist involvement. The bottleneck is not the technical complexity of the change but the absence of an interface for expressing task-level intent directly.

Ambiguity compounds this. Operator task descriptions are often incomplete or underspecified, so multiple robot behaviours may be consistent with a single instruction. Resolving this typically relies on trial-and-error or expert intervention, increasing reconfiguration time and cognitive load, particularly for non-experts who must simultaneously reason about intent, spatial context, and safety [8, 5, 6].

The specific problem addressed in this thesis is therefore the lack of task-level reconfiguration mechanisms that let non-expert operators express semantic task changes directly and manage the ambiguity inherent in natural language specification, while preserving reliable robot execution [7].

1.3 Aim and objectives

The overall aim of this thesis is to investigate how conversational interaction combined with vision-based perception can enable non-expert operators to perform semantic task-level changes in modular robotic manipulation systems operating under high variability. The work addresses the gap between human-friendly task specification and the structured, unambiguous representations required for reliable robot execution, with a particular focus on collaborative robotic pick-and-place applications.

From an industrial perspective, the aim is to remove the dependency on robotics experts when adapting robotic tasks after workspace or requirement changes, enabling operators to express task-level intent directly through natural language. From a research perspective, the thesis seeks to explore how task-level ambiguity can be managed through interactive clarification and contextual grounding, and how such mechanisms affect execution reliability and usability in practice.

To achieve the stated aim, the following objectives are pursued:

- Design a vision-based perception pipeline capable of detecting and localising objects in a reconfigurable pick-and-place workspace.
- Develop a conversational interface that supports multi-turn interaction and clarification questions to assist users in specifying and refining task configurations.
- Integrate perception data and conversational input to generate structured task-level configuration representations suitable for robotic execution.
- Develop a PDDL-based planning integration that takes the conversational output and generates verified, deterministic motion sequences.
- Develop a vision-grounded identity resolution mechanism that maintains configuration consistency when the physical workspace changes between sessions.
- Experimentally evaluate the system with respect to configuration correctness, the effectiveness of ambiguity detection and resolution through multi-turn dialogue, and the structural drivers of dialogue cost across tasks of varying complexity.

1.4 Research questions

To investigate conversational, vision-guided task configuration in modular robotic manipulation systems, this thesis addresses the following research questions:

1. How reliably does conversational interaction, supported by vision-based perception, produce correct and complete task configurations for robotic pick-and-place operations, and how does configuration correctness hold across tasks of increasing complexity?
2. How reliably does the system detect and resolve different types of ambiguity in conversational task specification, and does resolution effectiveness vary across ambiguity types?
3. What are the structural drivers of dialogue cost, and how do they scale with task complexity and ambiguity type?

1.5 Scope and limitations

This thesis focuses on task-level reconfiguration of robotic manipulation systems in the context of high-mix, low-volume production scenarios. The scope of the work is limited to modular pick-and-place applications executed by a collaborative robotic manipulator in a controlled laboratory environment. The primary objective is to investigate how conversational interaction, supported by vision-based perception, can enable non-expert operators to perform semantic task changes during task reconfiguration, rather than to optimise low-level robot motion or control performance.

Robotic platform and task domain. The platform is a single collaborative arm operating in a tabletop workspace with three reconfigurable object types: parts, containers, and kits. Tasks are limited to standard pick-and-place, where parts are grasped from one location and placed at another. The pick-and-place primitive is hard-coded and fixed throughout: the robot moves between a configured pick pose and place pose along a predefined motion structure. The system does not generate new motion primitives or adapt manipulation strategies. Only the pick and place poses change between operations.

Vision system. The vision system is restricted to detection and 2D localisation of objects within a known workspace. Advanced scene understanding, semantic segmentation beyond predefined classes, and learning-based perception are out of scope. Vision provides contextual grounding for task configuration rather than addressing perception robustness under adverse conditions.

Conversational component. The conversational component supports workspace reconfiguration through multi-turn dialogue over the high-level configuration attributes listed in Section 3.3. The agent operates within this structured configuration space and does not perform open-ended task planning. Reinforcement learning, end-to-end learning, and autonomous skill acquisition are explicitly excluded.

Evaluation and deployment context. The primary evaluation is performed by a single technically proficient individual who is also the system developer. A supplementary evaluation has one additional engineer (without prior exposure) perform Scenarios 2 and 3. The study is a capability assessment of the proposed system rather than a comparative study against industrial or proprietary programming environments. The implications of developer-as-evaluator are addressed in Section 6.1. Evaluation is conducted in a controlled laboratory setting, so long-term robustness, integration with production planning, industrial safety certification, mid-execution workspace changes, and advanced error recovery are not addressed. Accordingly, the thesis does not claim a general solution for arbitrary domains. The findings are specific to conversational task configuration for modular pick-and-place and are intended to demonstrate feasibility and characterise system behaviour rather than universal applicability.

1.6 Thesis structure

This thesis is structured as follows. Chapter 2 presents the related work and theoretical background relevant to the research, covering prior studies in high-mix manufacturing, human-centred robot programming, conversational interaction, and task-level robot reconfiguration.

Chapter 3 describes the research methodology and overall system approach adopted in this thesis. It outlines the research design, system architecture at the conceptual level, and evaluation strategy used to investigate conversational, vision-guided task configuration.

Chapter 4 details the design and implementation of the proposed system. This chapter describes the hardware and software components, including the perception pipeline, conversational interface, task configuration framework, and robotic execution setup.

Chapter 5 presents the experimental results and provides a discussion of the findings. The system is evaluated with respect to configuration correctness, ambiguity resolution quality, and conversational interaction structure across scenarios of increasing complexity.

Finally, Chapter 6 concludes the thesis by summarising the main contributions, reflecting on limitations, and outlining potential directions for future work.

2. Related work

This chapter surveys the bodies of literature most relevant to the thesis and identifies the research gaps the work targets. The review is organised thematically, progressing from robot reconfiguration and task-level programming through modular systems, human-centred interfaces, dialogue-based control, ambiguity resolution, vision-based grounding, and formal task planning.

2.1 Robot reconfiguration and task-level programming

The term robot reconfiguration carries several meanings: changes in arm configuration such as joint angles or kinematic structure, studied for reconfigurable or modular manipulators [9]; changes to the physical workcell, including tooling, fixtures, or layout; and runtime changes to software components, parameters, or their orchestration, as studied in software engineering [7]. This thesis understands reconfiguration at the task level, encompassing both mechanical and software reconfiguration, with a focus on adapting task semantics and execution logic through conversational interaction and vision-based grounding.

Traditional industrial robot programming is widely recognised as a limitation where tasks change frequently. Teach-pendant programming and manual re-programming require expert knowledge and time-consuming cycles of programming, testing, and validation whenever tasks or layouts change, making task changes slow and costly in high-mix scenarios and motivating more intuitive, higher-level approaches [10]. In response, human-centred robot programming research shows a clear shift from motion-centric toward task-centric paradigms in high-variability domains: rather than defining trajectories, waypoints, or joint motions, users specify what operation to perform and where, and the system derives the corresponding motions. Willibald et al. demonstrate this in robotic surface finishing, enabling non-experts to program through interactive task-focused workflows that abstract away low-level motion, reporting significant reductions in programming effort and cognitive load over motion-centric methods [11].

Central to such approaches are task-level abstractions, which encapsulate behaviour in terms of high-level tasks, goals, and parameters rather than explicit motion. By separating task intent from execution, they allow changes in variants, layouts, or priorities to be handled by modifying task descriptions instead of reprogramming trajectories, reducing reconfiguration effort while underlying primitives remain unchanged [11]. These abstractions are commonly realised through structured task representations such as state machines, skill-based systems, and behaviour trees, each trading off expressiveness, modularity, and maintainability. State machines offer clear execution semantics and simplicity but scale and maintain poorly as complexity grows. Skill-based systems decompose functionality into reusable low-level skills coordinated by an external mechanism. Behaviour trees combine hierarchical structure with modular control flow, with empirical studies showing improved mod-

ularity, reuse, and readability over state-machine approaches for complex missions while still supporting task-level coordination [12]. This separation lets tasks be encapsulated and reused as subtrees or composite states across missions, improving maintainability and scalability as complexity grows [11, 12].

Despite these advantages, many representations, including behaviour trees and hierarchical state machines, assume a largely predefined task structure fixed at design time, limiting execution to parameterisation and control-flow variation rather than structural change. Reconfiguration typically requires expert interaction with task models or configuration files, and adaptations are rarely expressed through intuitive, human-centred interfaces, leaving task-semantic reconfiguration indirect and cognitively demanding for non-experts [11, 12]. Task-level abstraction alone is therefore insufficient to reduce reconfiguration effort in practice: existing representations improve modularity and reuse but still require users to interact with predefined structures through technical mechanisms. This highlights the need for intuitive configuration interfaces that let non-experts modify task semantics (roles, priorities, parameters) at an appropriate abstraction level without altering underlying structures or motion primitives.

2.2 Modular robotic manipulation systems

Modularity in robotics describes designing systems as compositions of distinct, reusable components that can be exchanged or recombined for different tasks [13]. In industry it is often realised in hardware through modular workspaces and interchangeable elements such as grippers, toolholders, buffers, and sensors, supporting flexible high-mix, low-volume production via rapid physical reconfiguration without redesigning the system [13]. It is increasingly applied in software too: functionality is structured into reusable, composable components such as manipulation skills (parameterisable motion and control behaviours like grasping or placing), perception pipelines (sensing and processing steps that extract task-relevant information), and task representations (defining the sequence, dependencies, and parameters of actions), without modifying low-level control [13]. Together, hardware and software modularity improve flexibility and scalability and enable incremental adaptation to new tasks [14].

In industry, modular manipulation systems are most often realised as pick-and-place workcells underpinning sorting, kitting, packaging, and material handling [15]. These combine interchangeable hardware (end-effectors, containers, fixtures) with reusable software modules, enabling reuse across scenarios but increasing the number of configuration decisions that must be aligned across modules. As modularity expands the available modules and parameters, reconfiguration becomes more demanding: task objectives, object roles, spatial relationships, and execution parameters must be specified consistently, and changes to goals or layouts rarely affect a single component in isolation. Reassigning object roles or container functions, for example, may propagate to perception parameters, manipulation skill settings, and task sequencing logic. This raises cognitive and operational effort, especially under manual configuration, and makes cross-module consistency increasingly difficult as the configuration space grows [16, 7]. In current systems, although modules

are designed for reuse, their selection, parameterisation, and composition during reconfiguration remain largely manual, with consistency and dependency management left to system integrators rather than supported by the system [7]. Modular systems thus provide component-level reuse but lack high-level configuration interfaces that assist users in selecting, composing, and adapting modules, motivating approaches that shift effort from manual low-level alignment toward user-facing task specification and adaptation.

2.3 Human-centred robot programming and end-user programming

Human-centred robot programming prioritises human needs, abilities, and limitations over purely technical performance, aiming for programming, adaptation, and supervision that is intuitive, safe, and transparent [17]. It connects directly to end-user robot programming, which enables non-experts to configure behaviour through modalities such as demonstration and graphical interfaces rather than low-level code, reducing cognitive and technical burden, improving usability and trust, and supporting safe operation in changing environments [6]. Robot programming was traditionally done by experts (engineers, integrators) using low-level languages and detailed hardware, control, and architecture knowledge, but modern deployments increasingly require non-experts (operators, technicians, domain specialists) who understand the task domain yet lack formal robotics or software training [6, 5]. This shifts requirements: experts prioritise expressiveness and precise control, while non-experts need high-level abstractions that reduce training effort and cognitive load while preserving safety and robustness. The need for frequent task changes, costly under expert-dependent conventional programming, motivates letting end-users adjust behaviour directly to cut downtime and increase flexibility, a trend especially visible in collaborative robotics where robots are deployed close to workers and adapted to local workflows [6, 5].

Several approaches are used in practice. Teach pendants specify motions and waypoints through a structured interface but require training and are slow under frequent change. Kinesthetic teaching records trajectories by physically guiding the robot, reducing the need for abstract programming knowledge but requiring physical effort and compliant, safe control. Programming by demonstration generalises this by learning from demonstrations provided through guidance, observation, or interaction, leveraging user task knowledge while abstracting low-level control. Graphical interfaces provide visual representations such as flow diagrams, block-based programs, or menus, improving transparency of program structure but limited in expressiveness for complex tasks and often combined with other methods [6, 5]. In collaborative robotics these lower the programming barrier and support faster setup and reconfiguration, letting operators adjust behaviour without specialists for every change [6, 5].

In industrial practice, however, the dominant mechanism for pick-and-place cells (including collaborative ones) remains recipe-based management, where task programs are pre-authored and activated via HMI screens or PLC-backed panels. This

is effective for switching between known variants but offers no mechanism for modifying task semantics: changes to sorting logic, object roles, or priorities require a system integrator to manually update the underlying configuration [18]. The bottleneck is acute in high-mix, low-volume cobot deployments, where task changes are the norm but programming expertise is rarely on the shop-floor. As Fixl et al. note, real-world cobot solutions there are typically narrowly scoped or offer limited autonomy, and SMEs commonly lack resources for frequent expert reprogramming [19]. The research approaches above address parts of this problem, but the gap between expressive task-level intent and the configuration primitives exposed by industrial tools remains largely unresolved.

These limitations are most apparent in dynamic scenarios. Many approaches require repeated manual effort when parameters change, and programs become tightly coupled to their original context, reducing reuse and raising maintenance effort [6, 10, 5]. Cognitive factors compound this: end-users think in goals and outcomes while robots execute low-level sequences, so users must reason about constraints, execution order, and failure modes outside their task mental model, increasing cognitive load and authoring and debugging errors. The problem is amplified when interfaces expose mainly waypoints and action sequences without an explicit representation of task-level intent, forcing users to translate goals into detailed steps, making programs brittle under context changes and reducing transparency [6, 17, 10, 5]. This motivates a transition toward more expressive, intent-based interfaces that let users specify what should be achieved rather than how every step executes [6, 10, 5].

2.4 Dialogue-based and language-based robot control

Natural language has been investigated as a robot programming and interaction interface because it lets users express task-level intent in goal-oriented terms, in contrast to conventional programming that requires specifying low-level action sequences and platform constraints, which raises cognitive load and limits accessibility for non-experts [20]. By stating what should be achieved, natural language supports task-level representations that improve transparency and reduce brittleness under context change [20]. Language-based control spans a spectrum from command-oriented to dialogue-based interaction. Command-based systems map a single instruction to an executable behaviour or symbolic task structure, emphasising efficiency and minimal interaction [20]. Dialogue-based systems model task specification as an interactive process in which clarifications and feedback are exchanged over multiple turns, enabling grounding, error recovery, and refinement when information is missing or uncertain [21]. The distinction concerns how uncertainty is managed: up-front specification versus interactive repair. The system proposed here sits firmly on the dialogue-based end, treating task configuration as inherently interactive, using multi-turn clarification and vision-grounded context to iteratively resolve ambiguity and maintain a consistent symbolic world state across sessions.

Representative systems show how these models are instantiated. In industrial settings, natural language can act as a direct programming modality where instructions are parsed, grounded to objects, and translated into executable code, reducing manual teaching and low-level scripting [22]. In less structured environments, language is treated as a high-level intent description completed by inference: Nyga et al. interpret abstract, underspecified goals by integrating linguistic input with background knowledge and probabilistic reasoning to infer missing plan elements and, when needed, solicit information [23]. Language-guided control thus spans both instruction-to-code pipelines and intent-to-plan inference, with different trade-offs in interaction cost, robustness, and autonomy. Two advantages recur: natural language is expressive, capturing abstract goals and constraints without enumerating every step [23, 20], and familiar, relying on everyday communication rather than vendor-specific formalisms [20, 22].

These benefits carry intrinsic challenges in embodied execution. Natural language is ambiguous, underspecified, and context-dependent. Instructions permit multiple interpretations because references, spatial relations, and success conditions are not uniquely determined by text, so correct interpretation often requires combining language with perceptual evidence and task state [24]. Users also omit details they assume obvious, forcing robots to infer missing actions, objects, or constraints or to request clarification [23]. This motivates tightly coupling language processing with grounding and reasoning rather than treating language as a standalone command channel [24]. Deployment domain matters: domestic and service robots operate in open worlds with non-expert users, emphasising communication and dialogue [25], whereas industrial robots historically run in controlled workcells with stable processes and command-style interfaces [20], yet increasingly involve reconfiguration and variation, where predefined command vocabularies become limiting. Such vocabularies require designers to anticipate relevant actions, parameters, and variants in advance, restricting expressiveness and forcing system-specific phrasing that raises effort during modification and debugging [22, 20]. When tasks deviate from coverage, users must decompose intent into rigid sequences or resort to trial-and-error, and fixed vocabularies struggle with synonymy, abstraction, and omission [23]. Ambiguity resolution therefore becomes a primary requirement during reconfiguration, often demanding interactive mechanisms that query intent, use contextual evidence, and iteratively refine specifications rather than relying on one-shot interpretation [24].

2.5 Clarification, ambiguity and mixed-initiative interaction

Natural-language robot programming is inherently affected by ambiguity, because language leaves critical task aspects implicit and requires interpretation beyond literal meaning. Prior work identifies several complementary sources relevant to robotics. Object reference ambiguity arises when instructions refer to objects with underspecified or context-dependent descriptions, yielding multiple plausible referents due to perceptual uncertainty, attribute overlap, or similar objects [26]. Task

goal ambiguity arises when the desired end state or success criteria are underspecified, allowing multiple valid notions of task completion [27]. Execution constraint ambiguity, especially critical in industry, occurs when physical feasibility, safety, or process constraints are assumed but unstated, though they determine permissible actions [27]. These sources show that instructions rely heavily on shared assumptions, motivating systems that reason explicitly about ambiguity and interact when clarification is needed [26].

Linguistically, ambiguity manifests at multiple levels: lexical, where one word form has multiple meanings resolved through context [28]; spatial, where descriptions of location, orientation, or relation admit multiple interpretations from vague relations or alternative reference frames [29]; temporal, from underspecified ordering or reference points [30]; and intent-level, where a well-formed utterance supports multiple communicative goals and requires pragmatic inference [31]. These provide a theoretical foundation for why natural-language task specification is challenging. Building on this, this thesis adopts a reduced operational taxonomy targeting phenomena observable during configuration dialogue. The four operational types (referential, omission, implicit constraint, spatial), defined in Chapter 3, map onto the literature as follows: referential corresponds to object reference ambiguity [26] and to linguistic spatial ambiguity where a perspective-dependent description picks out multiple candidates [29]; implicit constraint maps to intent-level ambiguity requiring pragmatic inference to recover an unstated change [31, 27]; omission captures task goal ambiguity as missing required parameters [27]. Lexical and temporal ambiguity are not separate operational types because the configuration vocabulary is bounded and ordering is recovered from explicit priority rather than temporal expressions. A separate, system-specific challenge, matching newly detected parts to previously known parts after a rescan, is handled by a dedicated dialogue path described in Chapter 4 and is not one of the operational types.

To address ambiguity, many systems adopt a mixed-initiative paradigm in which both robot and user actively contribute to task specification rather than following one-shot command–execution. When understanding is uncertain, the robot asks clarification or confirmation questions instead of acting, making its hypotheses explicit so the user can confirm, correct, or refine, while the user adapts instructions in response. Task understanding emerges incrementally through dialogue. This bidirectional process is central to establishing common ground, especially in situated interaction where human expectations and robot perception often mismatch [32]. Within this paradigm, prior work explores clarification strategies spanning a spectrum. Confirmation questions, used when the robot holds a single dominant interpretation but lacks confidence, let the user validate or reject the hypothesis for efficient correction and incremental grounding [33, 32]. Disambiguation via alternatives, used when several interpretations are plausible, maintains competing hypotheses and asks targeted questions that contrast candidates to narrow the belief space [34]. Empirical studies show clarification has measurable impact: it resolves ambiguity iteratively until a complete, correct specification is obtained, increasing the likelihood of executing the intended task, and improves execution reliability by reducing uncertainty in grounded representations, yielding more consistent behaviour under perceptual noise or underspecification [34, 32]. The effect on trust is more

indirect. While rarely measured explicitly, clarification-based systems report higher usability and smoother collaboration. By exposing internal state and uncertainty, they help users form accurate expectations, commonly associated with increased trust and acceptance [34, 32].

2.6 Vision-based grounding and perception for task specification

Task-level abstractions, modular architectures, and dialogue-based interfaces all implicitly assume the robot can reliably interpret references to objects, locations, and relations in its environment. Without perceptual mechanisms anchoring linguistic and symbolic descriptions in the current scene, task-level representations remain detached from execution reality. Vision-based grounding addresses this by linking abstract specifications to concrete environmental states, enabling task-level systems to operate in dynamic workspaces. At its foundation lies object detection and localisation: perception pipelines identify relevant objects and estimate their spatial properties. In industrial pick-and-place, learning-based detection is often combined with hand-eye calibration to transform observations into executable grasp and placement parameters, performing reliably in structured environments [35], and recent advances show a shift toward learning-based methods robust under clutter, occlusion, and variation [36]. Beyond geometric localisation, object-centric representations decompose visual input into individual objects, improving robustness and generalisation in multi-object manipulation by enabling reasoning over object identities and relationships and isolating task-relevant entities from distractors [37]. Yet most systems still expose perceptual outputs as geometric or object-level data, leaving interpretation and configuration to predefined logic [38], motivating tighter perception-task integration.

Visual grounding directly maps linguistic references to concrete entities, essential for interpreting expressions such as “this object” or “the left bin”. Classical approaches treat this as referring expression comprehension using joint vision-language reasoning, with recent work showing grounding requires iterative reasoning and verification to resolve ambiguity when multiple candidates satisfy a description, explicitly modelling spatial relations, contextual cues, and inter-object dependencies [39]. Such methods are often evaluated in static image settings and remain decoupled from execution. More recent work leverages large vision-language models to extend grounding to open-vocabulary and context-dependent expressions without predefined categories [40], better reflecting natural task descriptions but introducing challenges of grounding precision, verification, and partial observability, leaving visual grounding an open problem in dynamic, safety-critical settings.

Perception plays a central role in reducing the ambiguity inherent in language-based control by constraining interpretation to the observable environment: grounding reduces referential ambiguity by identifying visually plausible candidates and reasoning over spatial and relational context [39], object-centric perception narrows valid interpretations by structuring scenes around discrete entities [37], and transforming observations into structured representations supports informed reasoning and pre-

vents premature or invalid execution [38]. Perception alone, however, cannot eliminate ambiguity, so integration with dialogue is essential. In situated interaction, mismatches between human intent and the robot’s perceptual model frequently cause dialogue failures during reference resolution. Empirical studies show perception errors significantly increase task breakdowns, while exposing the robot’s perceptual state through dialogue lets users adapt their language and resolve misunderstandings, demonstrating that perception actively shapes dialogue flow rather than serving as a passive upstream component [41]. Vision–language grounding supports this by enabling joint reasoning over visual and linguistic input [39], and VLM-based systems extend dialogue to open-vocabulary settings [40]. Still, perception-only approaches face fundamental limits: Representations are often incomplete or imperfect due to occlusion, noise, or misclassification. Even correctly detected objects do not reveal task-relevant roles, since multiple objects may match a description and functional roles such as target, landmark, or container are rarely encoded [37]. Vision–language models show that increasing perceptual coverage amplifies rather than removes the need for verification and interaction [40]. Vision should therefore be understood as a supporting capability rather than a replacement for interaction: it constrains interpretation and reduces ambiguity but cannot fully capture goals, preferences, or implicit intent, so robust task specification emerges from combining perceptual grounding with interactive refinement.

2.7 Classical task planning and automated sequence generation

While the preceding sections address how robots are programmed and reconfigured by operators, a complementary body of work approaches sequence generation from a formal, automated planning perspective. Classical AI planning derives action sequences from an explicit description of the world, available actions, and a goal state: rather than a human specifying each step, a planner reasons over a symbolic representation and produces a sequence guaranteed to achieve the goal, provided the world description is correct and complete [42]. The dominant formalism is the Planning Domain Definition Language (PDDL), a standardised representation language for AI planning competitions [42]. PDDL separates a problem into a *domain file* (object types, predicates describing world states, and actions with preconditions and effects) and a *problem file* (the concrete objects, their initial state, and the goal). Given these, a classical planner such as Fast Downward [43] applies search to find a valid action sequence from initial state to goal, producing optimal or near-optimal plans reliably and quickly for well-specified problems of moderate size.

In robotic pick-and-place, PDDL has been applied to sorting, kitting, and assembly tasks [44]. The domain file encodes the fundamental task structure (what it means to pick an object, what preconditions a placement requires, how roles constrain actions) and remains stable across configurations, while the problem file must be updated whenever the workspace changes: objects added or removed, containers reassigned, or priorities shifted. Because the planner reasons formally over the symbolic state, the generated sequence is guaranteed to satisfy all specified constraints, eliminating

an entire class of errors from manually authored sequences. Despite this, PDDL-based planning has seen limited adoption in high-mix, low-volume environments. The bottleneck is not the planner but the effort of maintaining the problem file: in static environments it is authored once and reused, but in dynamic environments it must reflect every change, requiring users who understand PDDL syntax, can map physical states to symbolic predicates, and can ensure logical consistency, a substantial barrier for non-experts [45].

Recent research explores large language models to automate problem file generation. LLM+P [45] translates a natural language task description into a PDDL problem file solved by a classical planner, then translates the plan back to natural language, establishing that LLMs can bridge informal descriptions and formal planning for well-structured domains. ViLaIn [46] incorporates visual input, using scene images to generate the object list and initial-state predicates, grounding the representation in perception, and Image2PDDL [47] similarly infers symbolic scene descriptions from images using vision–language models. These show perception and language models can substantially reduce problem-specification effort. However, a critical limitation runs through this work: all treat problem file generation as a largely autonomous, single-shot inference, producing a complete file from one image or text description without operator involvement. This fails in high-mix scenarios where task-relevant attributes cannot be reliably inferred from perception or text alone, since object roles, priorities, kit compositions, and operation modes reflect business logic and contextual knowledge residing with the operator rather than in the visual scene. A container identical to another may be an input source in one run and a defect collector in the next, invisible to any perception system. These approaches are also evaluated exclusively on planning correctness and do not measure the operator effort, time, cognitive load, and interaction complexity, required to supply the information, so reconfiguration effort is not addressed as a research outcome [45, 46]. This contrasts with the interactive, multi-turn approach adopted in this thesis, where the operator incrementally communicates workspace changes through dialogue and retains control over which semantic attributes are assigned.

This exposes a precise, practically significant gap. PDDL planners provide deterministic, verifiable sequence generation once a correct world state is available, and LLM-based interfaces can reduce the effort of expressing that state, but enabling a non-expert operator to efficiently communicate workspace changes, including high-level semantic attributes perception cannot supply, through natural, iterative dialogue has not been addressed. The present work targets this gap directly, using multi-turn conversational interaction to maintain the symbolic world state and feeding the resulting representation to a classical planner, rather than asking either the LLM or the operator to generate sequences manually. How task priorities are enforced inside the planning representation is a non-trivial design decision treated as an implementation concern in Chapter 4, together with the rationale for the choice adopted.

2.8 Summary of related research and identified research gaps

Across the reviewed literature, a clear trend emerges: industrial robot reconfiguration is moving from low-level motion editing toward adapting task semantics such as goals, roles, constraints, and dependencies. Task-level programming research shows that abstract representations separate intent from execution and improve readability, reuse, and scalability over trajectory-based programming [11, 12, 10]. Modular manipulation work shows that modular hardware and software architectures increase flexibility by making capabilities composable and replaceable [13, 14, 15]. Human-centred and end-user programming approaches reduce cognitive load by aligning interaction with how operators conceptualise outcomes rather than motions [6, 5, 17, 10]. Dialogue-based systems treat natural language as a means to express goals and constraints, with mixed-initiative interaction enabling clarification and repair when instructions are incomplete or ambiguous [22, 21, 32, 34, 26]. Vision and grounding research provides the perceptual basis by linking language to objects, locations, and states, including open-vocabulary reference [35, 36, 37, 39, 40]. Classical planning shows that PDDL planners generate deterministic, formally correct sequences from symbolic world states, with recent LLM-to-PDDL approaches assisting in translating informal descriptions into planning representations [45, 46, 47]. Critically, all existing LLM-to-PDDL systems treat world state specification as a single-shot autonomous inference, producing a complete problem file from a perception or text snapshot without operator involvement. This work departs from this by adopting an explicitly interactive, iterative approach in which the operator drives world state updates through dialogue across multiple turns.

Despite these advances, several issues remain unresolved in industry. Efficient task-level reconfiguration is still difficult: even with modular task representations, changes to structure or semantics often require expert intervention, and increased modularity enlarges the configuration space and creates cross-module consistency challenges [16, 7, 12]. Integration across dialogue, perception, and execution remains fragmented: many language-based systems assume reliable grounding and do not produce executable representations, while perception and grounding methods are often evaluated in isolation and do not address how uncertainty should influence interaction or execution [24, 38, 41, 40]. LLM-to-PDDL approaches address autonomous problem file generation but treat world state specification as single-shot inference, leaving unresolved the iterative maintenance of a symbolic world state under operator-driven changes involving attributes beyond perception [45, 46].

Taken together, this exposes a clear, practically significant gap. There is a lack of systems that support iterative, operator-driven maintenance of a formal world state representation across reconfiguration sessions in modular industrial manipulation, in particular preserving configuration consistency when semantic attributes such as object roles, priorities, and kit assignments cannot be re-inferred from vision alone, and when parts physically move or are replaced between sessions, invalidating previously observed state. Under these conditions vision provides only a partial, transient snapshot. The symbolic state a planner requires must be actively maintained through interaction, not reconstructed from perception on demand. Ex-

isting approaches provide individual building blocks (planners that generate correct sequences given a correct state, LLMs that translate informal descriptions into symbolic representations, and dialogue systems that resolve ambiguity through clarification), but no system unifies these into an end-to-end workflow in which an operator communicates incremental workspace changes through natural dialogue, the resulting symbolic state is preserved and updated across sessions, and a classical planner uses it to generate verified execution sequences [7, 24, 32, 39]. This work is therefore positioned as an integrated reconfiguration framework connecting perception, conversational interaction, and classical planning into a single operational loop [34, 41, 16], with evaluation focused on characterising system behaviour: how reliably the conversational approach produces correct configurations, how effectively it detects and resolves ambiguity, and how dialogue structure adapts to tasks of varying complexity.

3. Method

This chapter describes the research methodology, the system architecture at the conceptual level, and the experimental design employed in this thesis. It outlines the overall approach used to evaluate the proposed framework and explains how the experimental data is structured and interpreted. Detailed descriptions of the hardware, the software modules, and their internal workings are deferred to Chapter 4.

3.1 Research methodology

This study follows an applied experimental research methodology with a system evaluation design. The research is based on the implementation and controlled evaluation of a robotic pick-and-place system in which task configurations are specified through conversational interaction supported by vision-based perception. The goal of the methodology is to characterise how reliably the proposed system produces correct configurations, how effectively it handles ambiguity in natural language task specification, and how the conversational interaction adapts to tasks of varying complexity.

An experimental evaluation design is chosen over alternative methodological frameworks (such as design science research or case study methodology) because the research questions require systematic observation of system behaviour under controlled and reproducible conditions rather than iterative artefact refinement or contextual observation. By structuring the evaluation around scenarios of increasing complexity while holding all other system components constant, observed differences in system behaviour can be attributed to task characteristics rather than to confounding variables.

In this context, the independent variables are task complexity and the type of input ambiguity introduced. The ambiguity type is a controlled experimental input: each trial is constructed so that a specific ambiguity type or combination of types is present in the operator’s natural language phrasing. The four ambiguity types used in this thesis are defined in Section 3.4.2. The dependent variables are the measurable outcomes that characterise system behaviour: the correctness of the resulting task configuration, whether each induced ambiguity instance is resolved by the system, and the structure of the conversational interaction expressed as turn counts and turn-type distributions. Dialogue cost is reported internally (within and across scenarios of the same system) rather than against an external comparison such as expert reprogramming or a structured GUI. The evaluation therefore identifies the structural drivers of dialogue cost and how they scale with complexity and ambiguity type, rather than judging absolute efficiency relative to alternative reconfiguration approaches.

To ensure internal validity, several technical components are held constant across all trials. The same robotic manipulator, perception pipeline, motion planning framework, and underlying pick-and-place motion primitive are used throughout the evaluation. The pick-and-place primitive is hard-coded: it accepts a pick pose and a place pose, and the trajectory between them follows a predefined motion structure

that cannot be modified. The physical workspace itself is reconfigured between scenarios (containers, kits, and parts are rearranged or replaced as required by each scenario) and low-level attributes such as part position update automatically with the scene. This is a deliberate part of the evaluation rather than a confound.

3.2 Overall system architecture

The general system workflow is illustrated in Figure 1. At the top level the user can choose between three operational modes that depend on each other: *configure*, *plan sequence*, and *execute*. The intended order is to first configure the workspace as required, then plan a motion sequence, and finally execute it on the robot. After each mode the system returns to the top level, so the user can decide which mode to enter next.

When entering configure mode, the user chooses one of three sub-modes: *new config*, *update config*, or *edit config*. The new config sub-mode captures a fresh image and starts from a clean configuration file. The update config sub-mode loads the current configuration, takes a new image, and lets the user reconcile the current scan with the previous state before overwriting it. The edit config sub-mode loads a specific saved configuration from memory and allows direct adjustments to it without taking a new scan. All three sub-modes converge into the same multi-turn LLM dialogue, in which the user describes the desired changes in natural language. Once the user is satisfied, the resulting configuration is saved to memory.

The plan sequence mode loads the current workspace configuration and forwards it to a PDDL planner, which produces a motion sequence saved alongside the configuration. The execute mode loads that motion sequence and forwards it to the robot for execution. Once execution completes, the workspace has changed because the robot has moved parts. The configuration is updated automatically so that it again represents the post-execution state of the real world.

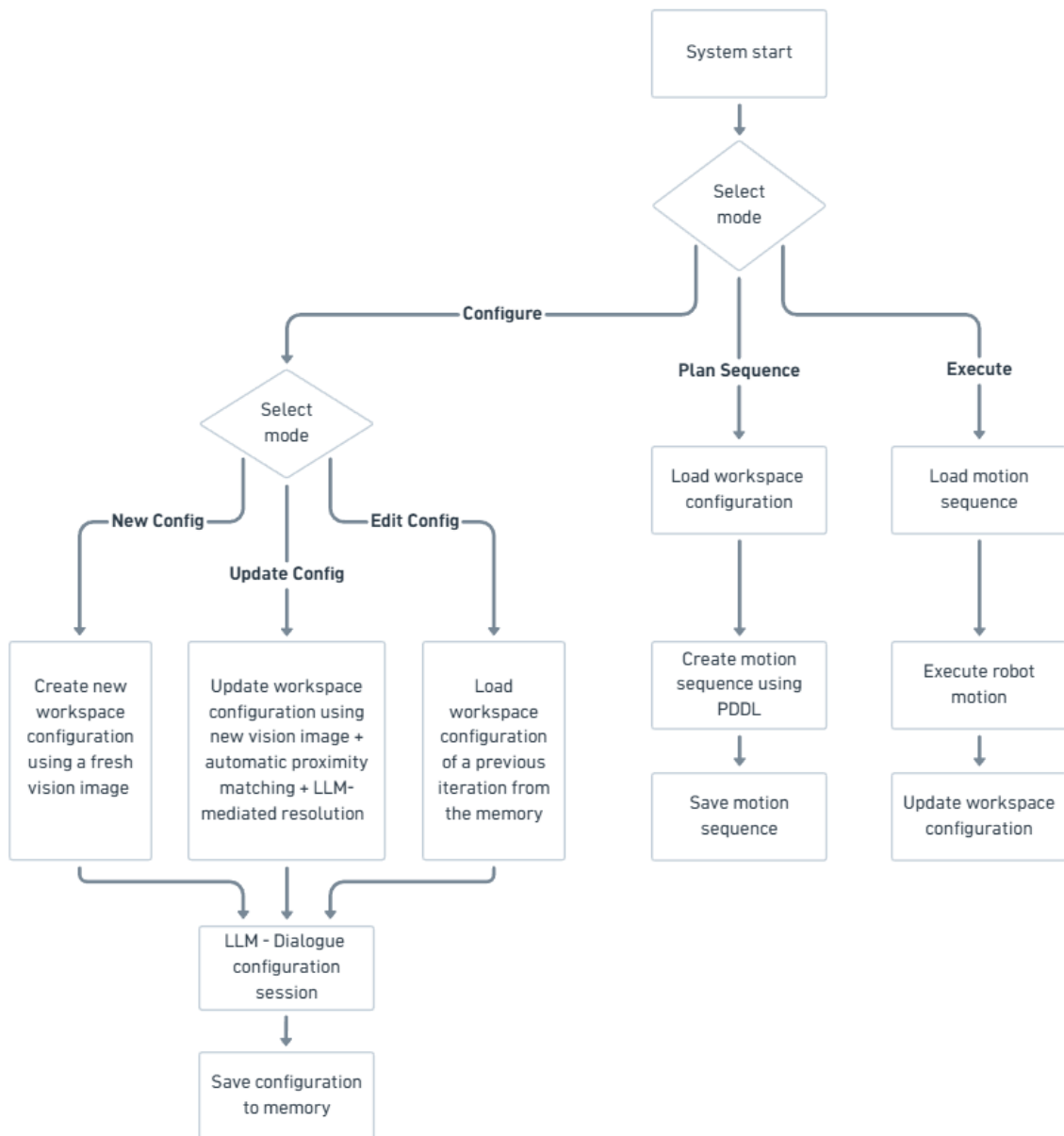


Figure 1: Operational session flow of the proposed framework.

The internal decomposition of the system is shown in Figure 2. Four functional modules together with a PDDL planner are coordinated by a top-level orchestration layer that handles mode selection and sequences calls between modules. The orchestration layer contains no task or LLM logic of its own. This separation ensures that the LLM and the PDDL planner have distinct, non-overlapping responsibilities. The LLM maintains and updates the symbolic world state through dialogue, while the planner consumes that state to generate verified action sequences.

Vision module. The vision module comprises a workspace-mounted camera and a perception pipeline for object detection and localisation. It captures the current state of the workspace and produces a configuration file describing each detected object together with its low-level attributes, that is, properties that can be determined directly from perception (such as position, orientation, size, and colour).

Communication module. The communication module is the primary interface between the user and the system. It implements the multi-turn LLM dialogue that handles all conversational interaction during workspace reconfiguration. Through this dialogue the user specifies high-level configuration attributes that cannot be inferred from perception alone. These attributes are introduced and listed in Section 3.3. The output of a dialogue session is a structured change record that is forwarded to the configuration module. The communication module additionally drives the LLM dialogue used during the update scene sub-mode, presenting automatic matching results to the operator and resolving any ambiguous identity cases interactively.

Configuration module. The configuration module maintains and updates all workspace configuration states. It applies change records produced by the communication module, manages the update scene pipeline by performing automatic proximity-based matching against the stored configuration, and derives post-execution states by applying completed motion sequences to the pre-execution configuration. All configuration states are persisted to enable iterative reconfiguration across sessions.

PDDL planner. The planner consumes the current configuration and produces a verified motion sequence that satisfies the encoded constraints. The choice of planner, the structure of the PDDL representation, and the rationale for how priority constraints are encoded are discussed in Chapter 4.

Execution module. The execution module commands the robot to carry out the pick-and-place actions defined in the motion sequence. After execution completes, control returns to the configuration module, which updates the workspace description to reflect the post-execution state, ensuring that any subsequent reconfiguration or planning session operates on an accurate representation of the physical workspace.

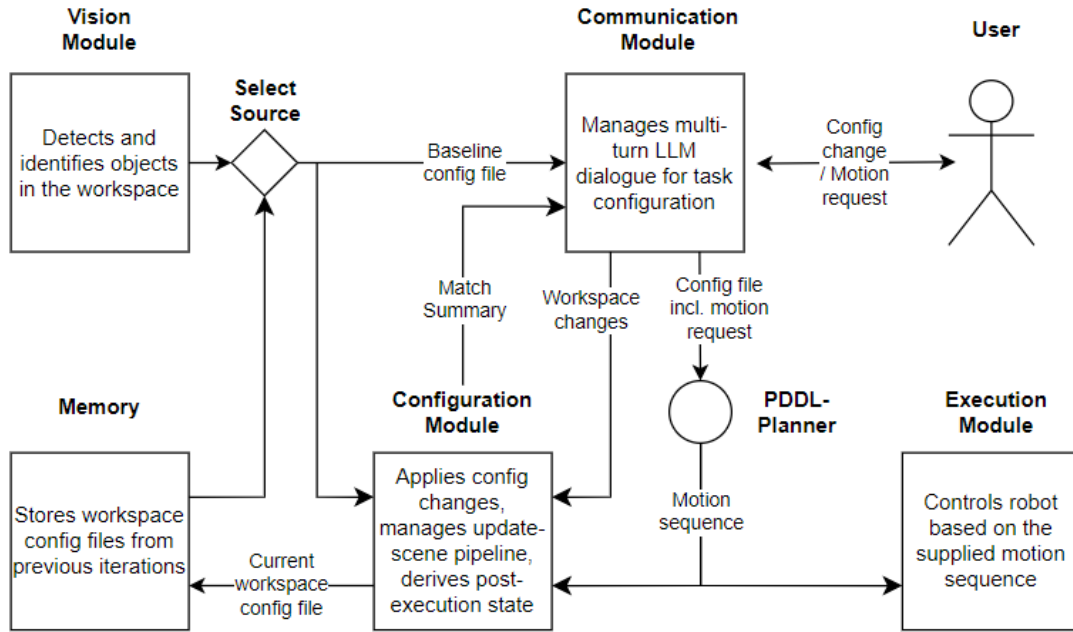


Figure 2: Overall system architecture of the proposed framework.

3.3 High-level configuration attributes

A central distinction in the system is between low-level and high-level attributes. Low-level attributes are properties that the vision module can measure directly, such as part position, orientation, and colour. High-level attributes describe semantic configuration decisions that the operator must specify and that no perception system can infer on its own. Table 1 lists the eight high-level attributes supported by the system. They form the configuration vocabulary used throughout the remaining chapters. Whenever a chapter mentions a specific attribute, the term refers to the corresponding entry in this table.

Attribute	Purpose
Operational mode	Selects the task type the system is currently configured for: <i>sorting</i> or <i>kitting</i> .
Object role	Assigns each container or kit a role of <i>input</i> (parts are picked from it) or <i>output</i> (parts are placed into it).
Kit recipe	Defines the contents of one or more kits as a list of (colour, quantity) entries that must be filled to complete the kit.
Part compatibility	Restricts which parts may be placed into which containers or kits, expressed as inclusion or exclusion rules over part properties (colour, fragility, name) and target receptacles.
Priority	Defines the order in which parts and receptacles should be processed. Pick priority orders parts; receptacle priority orders the order in which output containers or kits are filled.
Part fragility	Marks individual parts as fragile, causing the robot to handle them at reduced velocity throughout the pick-and-place cycle.
Batch size	In kitting mode, limits how many kits are filled within a single planning cycle, supporting incremental workflows.
Fill order	In kitting mode, specifies whether kits are filled <i>sequentially</i> (one kit completed before the next begins) or in <i>parallel</i> (kits are filled in interleaved fashion).

Table 1: High-level configuration attributes supported by the system.

3.4 Experimental design

The evaluation is structured into four scenarios of increasing complexity, addressing two distinct domains of ambiguity. *State-description ambiguity* arises when the operator describes physical workspace changes using imprecise or incomplete language, requiring the system to correctly update object identities and preserve previously assigned attributes. *Task-specification ambiguity* arises when the operator specifies intended task semantics using language that is underspecified, spatially relative, or semantically indirect. Scenario 1 establishes an unambiguous correctness baseline. Scenario 2 targets state-description ambiguity. Scenario 3 targets task-specification ambiguity. Scenario 4 combines both domains in a full end-to-end cycle. The detailed setup of each scenario, including workspace contents, exact trial structure, and number of repetitions, is given alongside the results in Chapter 5.

The operator interacts with the system in unscripted natural language, but the phrasing in each trial reflects the targeted ambiguity profile: trials with no induced ambiguity require the operator to formulate fully specified instructions, whereas trials with one or more induced ambiguity types allow correspondingly looser, more colloquial phrasing. All primary trials are performed by the system developer. In Scenarios 2 and 3, a single additional engineer with no prior exposure to the system performs one trial per scenario as supplementary evidence on interface accessibility.

Three metrics are recorded across all scenarios: *configuration correctness* (binary outcome per trial), *ambiguity annotation* (per ambiguity instance: type, whether the system detected it, the clarification question issued, and whether the instance was eventually resolved), and *dialogue structure* (turn count and turn-type distribution per trial). Whether an ambiguity is resolved within a single turn is not the success criterion. Some instances require multiple turns to converge, which is then reflected in the dialogue structure metric.

3.4.1 Per-type ambiguity evaluation

For scenarios involving ambiguity, the evaluation follows the per-type methodology established in prior work [26, 8], in which ambiguity types are defined taxonomically and each test input is constructed to trigger a specific combination of types. Within an ambiguity experiment, each repetition targets a different combination of ambiguity types while the intended configuration outcome remains identical across repetitions. Combinations progress from single-type or two-type pairings to progressively larger sets, with each type appearing in multiple trials. The targeted ambiguity types per trial are predefined. The natural language phrasing used to realise them is produced by the operator at evaluation time and is not scripted in advance. The ambiguity profile of each trial is reported in the per-scenario result tables of Chapter 5, and the complete raw and annotated dialogue logs are publicly available through the thesis repository [48].

3.4.2 Ambiguity type definitions

Each ambiguity instance encountered during a trial is classified into one of the four operational categories defined below. An ambiguity instance is a point in the dialogue at which the operator’s input does not uniquely determine the intended configuration change, requiring the system either to ask for clarification or to make an explicit assumption that the operator must confirm. The mapping of these four categories onto the linguistic and robotic ambiguity literature is given in Section 2.5. The short forms in parentheses are used throughout the result and discussion chapters.

- **Referential ambiguity (R):** The operator’s description refers to an object or container that cannot be uniquely identified from the description alone, because multiple objects in the workspace match the given attributes. Example: “the container on the left” when two containers could plausibly be described as being on the left.
- **Omission (O):** The operator’s instruction is missing a required parameter that the system needs to produce a complete configuration. The missing information is not implied by context and must be explicitly requested. Example: the operator specifies a kit recipe but does not state the batch size.
- **Implicit constraint (I):** The operator’s instruction implies a configuration change that is not stated directly but follows logically from the stated intent. Example: “the red parts are delicate”, which implies the fragility attribute should be set even though the word “fragility” is not used.
- **Spatial ambiguity (S):** The operator uses a spatial description (e.g. “the bin on the right”, “the part near the edge”) that admits multiple valid interpretations due to the workspace layout or the absence of a shared spatial frame of reference.

A separate, system-specific issue is the identity resolution problem encountered during the update scene sub-mode, where a newly detected part may not be unambiguously matchable to a previously known part. This issue is handled by a dedicated dialogue path, described in Chapter 4, and is not treated as one of the four operational ambiguity types in the evaluation.

3.4.3 Turn type definitions

Each dialogue turn is classified into exactly one of the following five categories. The short forms in parentheses are used throughout the result and discussion chapters.

- **User instruction (USR-INST):** The operator provides a new piece of task-relevant information or requests a configuration change. This includes the initial task description as well as follow-up instructions that introduce new requirements rather than responding to a system question.
- **System clarification (AST-CLAR):** The system asks a question to resolve missing, ambiguous, or underspecified information in the operator’s input.
- **User response (USR-CLAR):** The operator answers a system clarification question, providing the requested information rather than introducing a new requirement.
- **System proposal (AST-PROP):** The system presents a concrete set of proposed configuration changes and requests confirmation before applying them.
- **User confirmation (USR-CONF):** The operator confirms or rejects a system proposal.

A sixth label, **system constraint-violation turn (AST-VIOL)**, is used in the result tables to count turns in which the system flags a constraint violation rather than asking for clarification or proposing a change.

3.4.4 Results presentation

Results are reported per scenario and per trial. For each scenario, individual trial outcomes are presented alongside aggregated counts. Configuration correctness is reported as a table of per-trial binary outcomes with an overall success rate per scenario. Ambiguity annotations are reported in the same tables through the trial’s ambiguity profile and the resulting turn count and extra turn count. Dialogue structure is reported as turn counts per trial together with aggregated turn-type distributions per scenario in a cross-scenario summary table. Short interaction snippets are quoted in the discussion to illustrate specific behaviours. Complete dialogue transcripts, both raw and annotated, are publicly available in the thesis repository [48].

3.4.5 Constraint violation handling

In addition to ambiguity trials, Scenarios 2 and 3 include supplementary constraint violation trials. These present the system with instructions that are unambiguous in intent but violate a logical or real-world constraint of the configuration space, for example a kit recipe specifying more parts than fit in a kit. The system must detect the violation, inform the operator, and preserve a valid configuration state. Enforcing basic operational constraints is a prerequisite for a meaningful evaluation environment, so these trials are included as a practical complement to the ambiguity evaluation. Scenario 1 is excluded as it serves as an ideal-condition baseline, and Scenario 4 is excluded because the complexity of the full cycle makes it difficult to isolate constraint-related behaviour from other failure modes.

3.5 Sources of error and uncertainty

Several factors may influence the measured results. The vision system can produce incorrect part detections, colour misclassifications, or missed detections. If not caught during operator confirmation, an inaccurate initial scan propagates into downstream modules. This risk is greatest in Scenarios that rely on the update scene mechanism. The LLM is non-deterministic: even with identical input, it may produce different dialogue paths or require different turn counts to reach the same outcome, so consistent outcomes across varying paths indicate stable behaviour while divergent outcomes identify sensitive interaction patterns. User input variation is a deliberate feature of Scenarios 2 and 3, where the operator’s phrasing changes between trials to test sensitivity to phrasing. In other scenarios, minor unintentional differences cannot be fully excluded. Finally, a methodological risk arises from having the system developer act as the primary evaluator, since this introduces familiarity-based bias in phrasing and interaction style. The volunteer trials in Scenarios 2 and 3 partially expose this. The implications are discussed in detail in Section 6.1.

3.6 Experiment evaluation

The four scenarios are mapped to the three research questions as follows. Scenario 1 establishes the unambiguous reference for RQ1 and RQ3. Scenarios 2 and 3 address all three research questions: configuration correctness under ambiguity (RQ1), per-type ambiguity detection and resolution (RQ2), and turn-cost variation across ambiguity profiles (RQ3). The constraint violation trials in those scenarios contribute additional evidence to RQ1 by testing whether the system preserves a valid configuration state under logically invalid input. Scenario 4 evaluates correctness across a full multi-phase cycle (RQ1) and exposes how dialogue cost accumulates under multi-phase complexity (RQ3). The supplementary volunteer trials in Scenarios 2 and 3 contribute initial evidence on whether the clarification dialogue is interpretable without prior system knowledge, relevant to RQ2.

4. Design and Implementation

This chapter details the design and implementation of the system, covering the hardware platform, workspace fabrication, vision pipeline, communication and configuration modules, planner and execution setup, and user interface. The thesis codebase is publicly available on GitHub [48], while robot control relies on a separate repository [49] that provides infrastructure for experimental evaluation but is not part of the core thesis contribution. All Python scripts and their usage are listed in a table at the end of each section.

4.1 Hardware platform

The hardware configuration of this project was determined by the equipment available in the university laboratory. The manipulation system is based on a Franka Emika Panda robotic arm. As a collaborative robot, it provides inherent safety features and limited payload capacity, reducing the risk of damage to users and the workspace in the event of system failures or unintended collisions. This made it suitable for controlled experimentation on conversational task configuration without introducing industrial safety constraints. For visual perception, an Intel RealSense D435 camera is used, mounted on the robot tool flange in an eye-in-hand configuration. The camera was selected primarily because it was already mechanically integrated into the laboratory setup, ensuring stable and reproducible positioning relative to the workspace. A Linux PC with a real-time kernel controls the robot via the Franka Control Interface (FCI) using ROS Noetic together with the libfranka library. ROS 2 is not employed because the robot model only supports the older ROS version.

4.2 Physical workspace design and fabrication

The physical workspace forms the foundation of the system and its experimental evaluation, supporting robot pick-and-place operations across two distinct modes: kitting and sorting. Three object types were designed and manufactured for this purpose: parts, kits, and containers. The system targets experimental scenarios that roughly simulate high-mix, low-volume production cases, allowing for simplified object designs compared to industrial applications. All objects were fabricated using 3D printing, enabling rapid production and design iteration. The parts do not require high structural strength since they experience no mechanical stress during experiments. For vision system detection, AprilTags were printed on paper and affixed to kits and containers. A ChArUco board was similarly printed and mounted onto a 3D-printed board, screwed to the workspace in a fixed position. Its location relative to the robot base frame was measured and hardcoded into the system. Further details are provided in Section 4.3, and Figure 5 at the end of this chapter provides an overview of the complete workspace assembly.

4.2.1 Parts

All parts share an identical cylindrical shape, simplifying gripper design and approach programming. The only distinguishing feature is colour: blue, red, and green, simulating different product types. Six parts per colour were printed, allowing each container to hold a complete set. Figure 3 displays the printed parts.

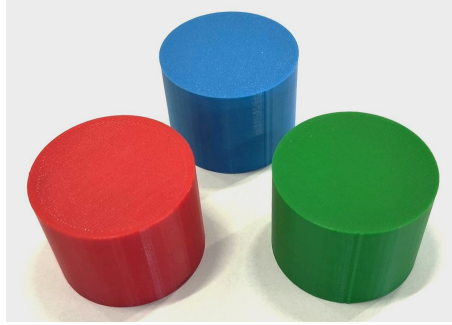


Figure 3: 3D-printed part variations

4.2.2 Kits and containers

Three kits were printed, each with capacity for three parts, enabling small-batch kitting operations and sized to balance workspace constraints with reliable robot placement accuracy. Containers simulate industrial part storage boxes. Picking from realistic configurations where parts are randomly positioned and stacked requires specialised vision and gripping systems beyond this thesis’s scope. A simplified design with predefined part slots was therefore adopted, maintaining experimental validity while ensuring consistent testing conditions. Figure 4 shows the printed kits and containers with AprilTags affixed.

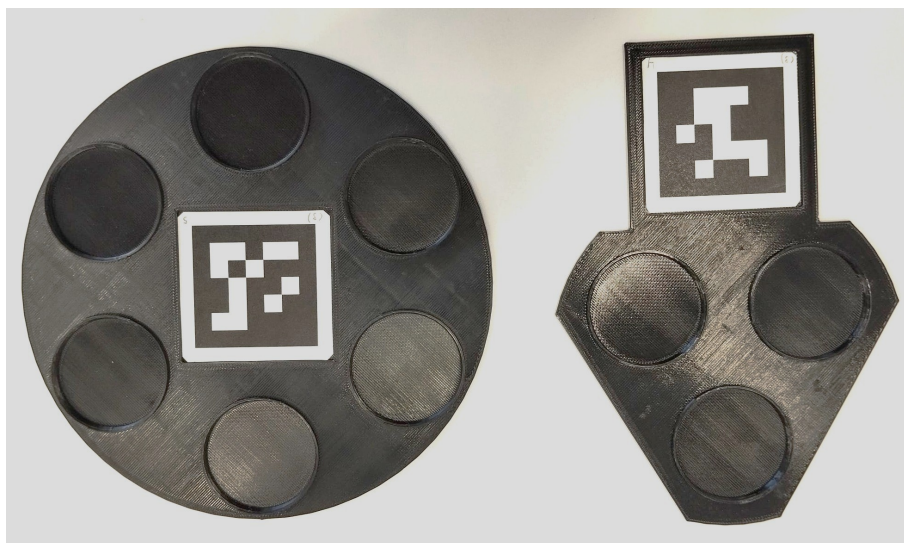


Figure 4: 3D-printed container (left) and kit (right).



Figure 5: The workspace used for experimental pick-and-place operations

4.3 Vision module

The vision system establishes a complete perception pipeline that transforms raw camera images into structured workspace representations for robotic manipulation. The pipeline consists of four primary stages: image acquisition, metric workspace calibration, object localisation, and data structuring for downstream planning modules. This architecture enables the system to detect parts, kits, and containers in the workspace and provide their precise 2D positions in the robot base frame.

4.3.1 Camera configuration and image acquisition

The system employs an Intel RealSense D435 camera configured to capture 1920×1080 resolution colour images. The camera mounts on the robot gripper and operates from a fixed overhead position that ensures repeatable imaging geometry. This position was recorded during initial setup using the “Franka Demo” repository [49].

Image acquisition uses a subprocess-based architecture to isolate potential library conflicts. The system captures frames after a warmup period to ensure stable automatic exposure and white balance. Manual camera settings can be specified when consistent lighting conditions require fixed parameters.

Captured images undergo lens distortion correction using factory-calibrated intrinsic parameters. An ROI cropping step removes workspace boundaries and environmental clutter. This preprocessing ensures that subsequent detection algorithms operate on clean, geometrically corrected imagery.

4.3.2 Metric workspace calibration

The workspace uses a fixed ChArUco calibration board as the reference frame. A ChArUco board was selected instead of a standard ArUco marker due to its higher pose estimation accuracy resulting from the combined use of chessboard corners and fiducial markers [50]. Detection algorithms locate the board pattern and compute a homography matrix that maps between board coordinates and image coordinates. This transformation enables precise metric measurements by converting pixel locations to real-world positions on the board plane.

The workspace origin is defined at the board corner. A fixed offset relates this reference frame to the robot base frame. This offset was measured once during setup and remains constant throughout operation. The fixed-board approach is justified for this experimental system since the workspace geometry and camera position remain static while only object arrangements change. Production deployment could employ automated hand-eye calibration instead.

4.3.3 Container and kit localisation

Kits and containers are identified through AprilTags affixed to each object. AprilTags were selected because they are widely used in industrial and research contexts and enable reliable and straightforward detection and identification [51]. The system defines distinct tag groups for different object types and maps them to sequential numbering for user-facing displays. Tag detection runs in isolated processes to prevent potential failures from affecting the main vision pipeline.

For each detected tag, the system computes its centre position in board coordinates and transforms it to robot coordinates. Tag orientation is extracted from the detected marker geometry and converted to rotation angles for gripper alignment.

Each kit and container has predefined slot positions stored as local offsets from the tag centre. These offsets are transformed according to detected tag orientation and translated to workspace coordinates. The resulting absolute positions define all available slots. Slot orientations are provided to enable collision-free manipulation in densely packed arrangements.

4.3.4 Part detection pipeline

Part detection employs a multi-stage colour-based segmentation pipeline that processes images independently from tag detection. The approach is designed to handle uneven lighting, metallic surface reflections, and patterned backgrounds.

Raw images first undergo Gaussian blur for noise suppression [52], followed by Contrast-Limited Adaptive Histogram Equalisation (CLAHE) applied to the luminance channel in CIE-Lab colour space [53]. Using Lab space rather than HSV ensures that illumination is normalised without distorting the hue information subsequently used for segmentation. Segmentation operates in HSV colour space with per-colour adaptive thresholds. Each colour has individually tuned ranges experimentally determined for the laboratory setup. Binary masks undergo morphological processing to remove noise and reconnect regions fragmented by background patterns.

Part validation combines multiple geometric and photometric checks. Circularity is verified using both raw contour and convex hull metrics to handle irregular boundaries on patterned surfaces. HSV-classified regions are cross-validated against original colour values to reject achromatic regions and reflections. Since all parts share identical dimensions, undersized detections are rejected as spurious. Final positions undergo perspective distortion correction to reduce localisation error across the workspace.

4.3.5 Data structuring and output

After all objects are detected and localised, the vision system constructs a hierarchical workspace representation. Parts detected near kit or container slots are automatically assigned to those slots, while unassigned parts remain as standalone objects. Each object receives a unique identifier following consistent naming conventions, with parts numbered sequentially starting from Part_1 and ordered by spatial position to ensure consistent labelling across captures. Because these identifiers are assigned anew on each perception cycle rather than being physically tracked, the same physical part will receive a different identifier if it has moved between scans or if the set of visible parts has changed. This identifier instability across rescans is the primary motivation for the update scene sub-mode.

The workspace state is serialised to JSON format containing object lists, slot ownership mapping, predicates, and metric data for robot execution. Each object's metric entry includes position in robot frame, orientation, and grip rotation angle. JSON was selected as the intermediate configuration format primarily due to its hierarchical structure, which is well suited for structured large language model input [54]. Vertical position data is hardcoded for the experimental evaluation rather than derived from vision measurements.

The vision module produces three output artefacts: the workspace state JSON consumed by planning and execution modules, an annotated visualisation image displaying all detected objects with labels and coordinate markers, and a coordinate mapping file enabling efficient updates when configurations change. This structure decouples computationally expensive vision processing from lightweight configuration updates during interactive sessions.

4.3.6 Module components

Script Name	Description
Vision_Main.py	Main entry point that orchestrates the complete vision pipeline
pipeline.py	Coordinates all detection stages and produces final workspace state
vision_realsense.py	Interfaces with Intel RealSense camera for image acquisition
vision_capture_worker.py	Subprocess worker for camera capture to prevent memory corruption
vision_charuco.py	Detects ChArUco calibration board and computes homography matrix
vision_apriltag.py	Detects AprilTags on kits and containers
_apriltag_worker.py	Subprocess worker for AprilTag detection to isolate segmentation faults
vision_circles.py	Detects parts using colour-based segmentation and shape validation
geometry.py	Provides coordinate transformation and perspective correction utilities
assign_parts.py	Assigns detected parts to slots based on proximity
workspace_state.py	Constructs and serialises the final workspace state to JSON
config.py	Stores configuration parameters and hard-coded workspace offsets
io_jsonl.py	Handles JSONL file operations for logging detection results

Table 2: Vision module scripts.

4.4 Communication module

The communication module implements an LLM-based conversation system that translates natural language instructions into validated workspace configurations. The module operates as an intermediary layer between user intent and system execution, handling dialogue management, ambiguity resolution, constraint validation, and structured output generation. The architecture separates pipeline orchestration from LLM conversation, enabling cancellable operations and modular testing of individual components.

GPT-4 was selected as the underlying language model due to its strong instruction-following capability and native support for structured JSON output. It has also seen widespread adoption in comparable robotics language-grounding research [24, 55, 56]. A cloud API approach was adopted rather than local deployment, as the laboratory hardware is committed to real-time robot control. Lightweight local models have additionally demonstrated significantly lower task completion performance in comparable settings [57].

4.4.1 System prompt engineering

The system employs a structured prompt that prescribes conversation flow while enabling natural language interaction. The prompt architecture separates structural requirements from conversational tone through three distinct layers: conversation structure, data model specification, and output formatting.

The conversation structure layer specifies mandatory dialogue flows including when to request clarification, when to output structured data, and when to seek confirmation. Central to this layer is a processing hierarchy that guides the LLM through progressively broader interpretations of user requests. The hierarchy first attempts single-attribute matching, then handles multi-attribute requests, and finally addresses complex setup requirements.

This prioritisation minimises unnecessary dialogue overhead. When users state simple configuration changes, the system proposes only relevant updates without questioning unrelated attributes. The hierarchy recognises that users typically configure workspaces incrementally through focused instructions rather than complete declarations.

The data model layer describes the workspace schema with precise specifications for each attribute. The prompt defines allowed value sets, attribute dependencies, and semantic constraints that govern valid configurations.

The prompt also enforces strict ambiguity rules that prevent implicit assumptions. When instructions are unclear, the LLM must request clarification. Scene narration is explicitly forbidden except when explaining impossibility or responding to explicit description requests. This keeps interactions focused on configuration changes rather than state recitation, as users have visual feedback through the GUI.

4.4.2 Conversation flow management

The main conversation loop implements a state machine that tracks pending proposals and accumulated changes across multiple confirmation cycles. The loop alternates between LLM generation, structured output extraction, validation checks, and user response handling.

Each conversation turn begins with an LLM call that produces natural language explanation accompanied by optional structured updates. When a pending proposal exists, user input undergoes intent classification to determine whether it represents confirmation, rejection, or a new instruction. Confirmation triggers validation followed by accumulation into session state. Rejection discards the proposal. New instructions are processed after resolving whether they supersede or extend pending proposals.

This approach enables iterative configuration refinement through focused dialogue turns while maintaining session continuity across multiple interactions.

4.4.3 Ambiguity detection system

The ambiguity detection system operates as a background analysis layer that runs in parallel with the main conversation. When users submit instructions, the system spawns a separate LLM call with a specialised prompt that analyses for ambiguity types including referential ambiguity, missing parameters, implicit constraints, and spatial ambiguity.

Detection receives the full scene representation, recent conversation history, and the current message as input. Analysis runs at zero temperature to ensure deterministic classification. The system filters trivial inputs including confirmations and rejections to avoid unnecessary overhead.

The detection prompt explicitly respects single-attribute changes as non-ambiguous. When messages clearly map to changing one independent attribute, the analyser returns no ambiguity even if other attributes seem implied. This prevents false positives that would slow incremental configuration patterns.

When ambiguity is detected, the system formats the analysis into a hidden message that guides the main LLM toward targeted clarification. This injection occurs before the next LLM call, ensuring full context without requiring repeated detection passes.

4.4.4 Tool-augmented conversation

The LLM can invoke deterministic tools during conversation through function calling. Tools perform validation of operations and generate workspace summaries, enabling grounded responses based on actual state rather than inference.

Tool calls execute synchronously within the conversation loop. When the LLM emits a tool-call message, the system executes the corresponding function and injects the result. The LLM then incorporates this result into its next generation, ensuring responses reflect accurate workspace state.

4.4.5 Block parsing and validation

Structured output extraction employs pattern-based parsing with fallback strategies to handle formatting variations. The parser accepts multiple conventions while enforcing strict attribute schemas that reject unknown keys and validate value types against allowed sets.

A preprocessing step merges duplicate entries that occasionally appear when the model proposes related changes separately. This prevents silent data loss during parsing. Parse errors trigger immediate feedback insertion into conversation history describing the error and reminding the LLM of formatting conventions. Conversation continues with reformulation rather than termination.

4.4.6 User intent classification

User responses to pending proposals require intent classification to distinguish confirmations, rejections, and new instructions. The system employs a two-stage strategy combining deterministic pattern matching with LLM fallback for ambiguous cases.

Fast-path classifiers handle canonical responses through word boundary matching. Detection patterns identify session-ending keywords, confirmation words, and rejection words. The system handles compound confirmations that contain both acceptance and additional instructions, processing the confirmation immediately and feeding the extracted instruction back as a new message.

When pattern classification proves inconclusive, the system invokes a specialised LLM call with minimal context. If classification remains uncertain, the system directly prompts users to clarify their intent. This three-tier approach prioritises speed for common cases while ensuring accuracy for ambiguous responses.

4.4.7 Conflict detection and resolution

Conflicts arise when attributes receive new values that differ from earlier assignments within the same session, or when new rules contradict previously established constraints. Conflict detection operates on accumulated and pending changes only, not on the base configuration state. This design enables users to change previously configured attributes without triggering conflicts, as long as they do not contradict changes made earlier in the same session.

When conflicts are detected, the system presents each conflict with old and new values clearly identified. Users describe their resolution intent in natural language, which the system passes to an LLM for interpretation. The LLM generates a fully resolved update incorporating the user’s decision alongside non-conflicting changes from both sources.

This approach enables flexible resolution without requiring users to understand internal representation. The resolution LLM receives both the conflict list and the resolution instruction, producing a merged update that eliminates all conflicts.

4.4.8 Output structuring

The communication module produces two primary output artefacts: a “changes” file that accumulates all confirmed attribute updates, and a configuration memory that preserves timestamped snapshots for session continuity.

Changes accumulate through attribute-specific merge strategies. Some attributes replace previous values entirely to maintain semantic consistency, while others extend previous values additively when composition is appropriate. Attributes can also merge at finer granularity, allowing independent updates to different properties without requiring complete restatement.

The merge semantics handle edge cases including null values and empty collections, which clear attributes rather than extend them when encountered.

These strategies enable iterative refinement where users build complex configurations through multiple confirmation cycles without restating unchanged attributes, while maintaining a coherent final state that can be applied atomically.

4.4.9 Update scene dialogue

The update scene dialogue represents a specialised conversation mode that handles part identity preservation when rescanning the workspace. After the vision module detects parts in a fresh image, the system must determine which detected parts correspond to previously known parts and which represent new additions.

The dialogue prompt provides comparison between old and new workspace states, including lists of disappeared parts, new parts, and auto-matched parts. Auto-matching occurs deterministically based on spatial proximity and colour consistency, establishing tentative mappings that users can override.

The LLM uses a submission tool that accepts conversational messages and mapping dictionaries. The dialogue accumulates confirmed mappings across multiple turns, enabling incremental corrections. The system validates mappings to prevent duplicate identity conflicts.

Tool-based output provides stronger structural guarantees than code blocks. The function schema enforces required fields and type constraints at the API level, while the conversational message parameter enables explanation while maintaining separation between dialogue and structured data.

4.4.10 Cancellation mechanism

The conversation loop supports cancellation through event-based signalling that operates across all dialogue modes. The GUI layer sets an event when users trigger cancellation. The conversation loop polls this event at multiple checkpoints including before LLM calls and after tool executions.

LLM calls run in background threads with periodic cancellation checks. When the event is set, the thread receives a cancellation exception that propagates to the conversation entry point. The session handler catches this exception and returns partial results accumulated before cancellation, preserving progress without forcing completion.

The cancellable implementation spawns requests in daemon threads and polls the completion event at regular intervals. This balances responsiveness against computational overhead, enabling users to exit lengthy operations without system hang.

4.4.11 Module components

Script Name	Description
api_main.py	Main conversation orchestration and LLM interaction loop
prompts.py	System prompt templates for reconfiguration and update dialogues
block_parsing.py	Extracts and validates structured output blocks from LLM responses
ambiguity_detection.py	Analyses user input for referential and spatial ambiguities
user_intent.py	Classifies user responses as confirmations, rejections, or new instructions
change_management.py	Detects and resolves conflicts between pending and accumulated changes
capacity_tools.py	Validates workspace capacity and generates scene descriptions
scene_helpers.py	Utility functions for workspace state queries and position labelling

Table 3: Communication module scripts.

4.5 Configuration module

The configuration module manages workspace state transformations throughout the system lifecycle. Its central role is to maintain a coherent representation of the workspace as parts move, scans are repeated, and the operator changes the configuration. This is achieved by separating what the vision module can observe directly (the low-level attributes introduced in Section 4.3) from the high-level attributes the operator must specify, which are listed in Table 1. From this separation, the module derives three responsibilities: applying confirmed configuration changes, reconciling a fresh vision scan with the previously stored state, and updating the configuration after the robot has executed a motion sequence.

4.5.1 State structure

The workspace state is stored as a single PDDL-compatible JSON document, organised so that observations and intentions are kept distinct. An *objects* section enumerates the concrete entities present in the workspace. A *predicates* section combines low-level facts produced by vision (which part occupies which slot, which slots are empty) with high-level facts contributed by the operator (the high-level attributes from Table 1). A *metric* section stores the precise pose data the robot needs to execute motions, and a *workspace* section captures global, operation-level parameters such as the operational mode and batch size. Vision is allowed to overwrite low-level predicates and metric data on every scan. High-level predicates are only changed through the conversational dialogue.

4.5.2 Applying configuration changes

When a reconfiguration dialogue completes, the communication module produces a structured change record. The configuration module integrates this record into the current state, leaving low-level observations untouched. Different attribute types are handled in different ways: workspace-level parameters are merged in place, so independent attributes such as operational mode and batch size can be updated without touching one another, whereas list-valued attributes such as priority rules, kit recipes, and compatibility rules are replaced wholesale, since a new specification of these attributes is intended to define the complete set rather than extend the previous one. Object-scoped changes, such as a role assignment on a specific container or a fragility flag on a specific part, are applied by locating the target object and updating or creating the corresponding entry. Names referenced in the change record are validated against the current object list, and references to non-existent objects are skipped with a warning rather than aborting the entire update.

4.5.3 Scene update pipeline

A fresh scan is processed in two stages. First, the configuration module attempts to match newly detected parts to parts that were already present in the stored configuration. The first pass uses slot identity: when a fresh detection occupies the same slot as a part of compatible colour in the previous configuration, the two are treated as the same part. The second pass handles parts that have moved between scans or were never assigned to a slot, by computing distances between unmatched fresh and unmatched old parts of compatible colour and greedily pairing the closest. Pairs whose separation exceeds a small workspace-scale tolerance are not matched automatically. Genuinely new parts that cannot be paired with any prior part receive fresh sequential identifiers. Previously known parts that are not detected on the new scan are kept in the configuration with their last known attributes, so that temporarily occluded parts are not lost.

Once automatic matching is complete, the configuration module passes a summary of matched, new, and missing parts to the communication module, which conducts the LLM-mediated identity-resolution dialogue with the operator (described as part of the communication module). The configuration module then applies the confirmed identity mapping to the fresh state and merges the preserved high-level attributes from the previous configuration onto it. The result is a single state that combines fresh perceptual observations with the operator’s existing high-level decisions. This implements the core principle that vision determines physical reality and the dialogue determines behavioural intent.

4.5.4 Post-execution state update

After the robot has executed a motion sequence, the configuration is brought into agreement with the new physical state by replaying the executed sequence symbolically: each pick-and-place step empties its source slot, fills its destination slot, and updates the affected part’s stored pose. Slotted parts and standalone parts are handled through the same logic. Steps that reference invalid target locations are skipped rather than allowed to corrupt the state. Optionally, a post-execution rescan can be triggered automatically: it follows the same matching logic as the interactive update but does not invoke the operator. Detections that do not correspond to any known part are discarded as spurious, and parts that were expected but not detected (for example because of temporary occlusion) retain their predicted positions derived from the executed sequence. The merged result is saved as the new current configuration.

All state writes use atomic file operations (described in Section 4.8.3 of this chapter), guaranteeing that the on-disk configuration always reflects either a complete old or a complete new state. Each significant transition is also archived as a timestamped snapshot, so that the operational history of a session can be reconstructed for debugging.

4.5.5 Module components

Script Name	Description
apply_config_changes.py	Applies user configuration changes to workspace state predicates
update_scene.py	Manages vision rescanning with part identity preservation and state merging
apply_sequence_changes.py	Updates workspace state after robot execution based on motion sequence

Table 4: Configuration module scripts.

4.6 Planner module (PDDL planner)

The planning module transforms a workspace configuration into an executable motion sequence using automated task planning. It consumes the PDDL-compatible state produced by the configuration module and produces an ordered list of pick-and-place operations that satisfies the configured goals, role assignments, and compatibility rules. Fast Downward serves as the planning backend.

4.6.1 Planner selection and priority enforcement

Fast Downward was chosen because it supports the PDDL 2.1 features needed in this work [43], in particular derived predicates and action costs, which allow priority ordering to be enforced directly inside the domain. Hard constraint enforcement is adopted in this thesis: priority is encoded as preconditions in the planning domain so that any plan returned by Fast Downward is, by construction, consistent with the operator’s priority specification. Encoding priority as a soft post-processing step would have been simpler, but it would forfeit the formal guarantees that motivate using a classical planner in the first place, and it can produce reorderings that are no longer feasible under the remaining task constraints. With the experimental workspace sizes used here, planning completes well under one second per call.

4.6.2 PDDL domain architecture

The domain defines two core actions, *pick* and *place*, together with the predicates and types that describe their preconditions and effects. Pick removes a part from a slot of an input receptacle. Place moves the held part into a slot of an output receptacle. Their preconditions enforce that parts can only leave input locations and only enter output locations where they are compatible. Kits and containers are modelled as subtypes of a single receptacle type so that the planner can distinguish their roles during search.

Priority is encoded by splitting each pick action into priority-specific variants and gating each variant on a derived predicate that holds only when no higher-priority part remains available. Place actions are split analogously for receptacle priority. To avoid dead-locks caused by stock parts that exceed what the goal requires, only goal-bound parts receive priority tags during problem generation. The rest remain untagged and can be picked at any time. In multi-kit kitting, priority checks are additionally scoped to the kit currently being filled, so that high-priority picks reserved for later kits do not block the current one. Action costs complement these hard constraints by biasing the search towards preferred orderings when more than one valid plan exists.

4.6.3 Problem generation

A PDDL problem file is generated from the current configuration and the chosen operational mode. The objects section enumerates kits, containers, parts, and slots. Standalone parts that are not assigned to a slot receive a virtual slot in a special container so they can be treated uniformly during picking. The initial state populates predicates for part-slot relationships, slot occupancy, role assignments, colours, and priority tags for goal-bound parts. Compatibility predicates are produced from the operator-defined rules in two passes: inclusion rules first, exclusion rules afterwards so that exclusions always dominate, supporting expressions such as “green parts allowed in kits, but fragile green parts not allowed in Kit 1”.

The goal is then derived from the operational mode. Sorting goals require that every input part be moved to some compatible output slot, leaving the choice of slot to the planner. Kitting goals are derived from the kit recipes: for each (kit, colour, quantity) entry, the highest-priority compatible parts up to the requested quantity become explicit goal placements. Universal recipes are expanded across all output kits, and the batch-size attribute limits how many kits receive goal entries within a single planning cycle.

4.6.4 Fast Downward integration

Fast Downward runs as an external subprocess. It is invoked with A* search using the FF heuristic, a standard goal-distance estimator from the planning literature that is well suited to single-agent classical planning problems of the size considered here. Goal reachability is validated by Fast Downward’s translator before search begins, so configuration errors such as insufficient available parts or incompatible role assignments are detected up front. Domain and problem files are written to a temporary directory, the planner is invoked, the resulting plan is parsed if planning succeeded, and the directory is removed. Debugging options can retain the files when needed.

4.6.5 Plan parsing and sequence interface

The plan returned by Fast Downward is converted into the same pick-and-place sequence format used by the execution module. The parser walks the plan, pairs pick actions with their corresponding place actions, strips the priority suffix from action names, restores the original mixed-case object names, and removes the virtual slots used for standalone parts. The planner module exposes this functionality through a single entry point that handles state-to-PDDL translation, planner invocation, and sequence construction, so that the execution module can request a sequence without depending on planning internals.

4.6.6 Module components

Script Name	Description
pddl_planner.py	Generates PDDL domain and problem files, invokes Fast Downward, and parses plans into motion sequences
planner_interface.py	Provides unified entry point for sequence generation and orchestrates the planning pipeline

Table 5: Planning module scripts.

4.7 Execution module

The execution module translates motion sequences into physical robot motion. It consumes the pick-and-place sequence produced by the planner and drives the Franka Emika Panda through ROS and MoveIt.

4.7.1 Robot control foundation

Low-level robot control is provided by a separate repository (*Franka Emika Panda Demo*) developed before this thesis [49]. That repository contains the ROS Noetic workspace, the MoveIt configuration, the Pilz Industrial Motion Planner integration, and a robot interface class that exposes high-level motion primitives. It also includes the interactive teaching tools used to record the static workspace poses used in this work. The repository is kept separate from the thesis codebase so that the control stack and the conversational/planning stack can evolve independently. The remainder of this section describes the components built on top of this foundation specifically for the experimental system.

4.7.2 Pick-and-place cycle

Each pick-and-place operation follows the same hard-coded motion structure. The robot starts at a pre-taught *Home* pose, moves quickly to a position above the configured pick location, aligns the gripper with the part's grip angle, and descends linearly to the part. After the gripper closes, the robot retreats vertically, returns to Home, moves above the destination slot, descends linearly into the slot, releases the part, and again retreats vertically before returning to Home. Only the pick pose and the place pose change between operations. Everything else, including the Home pose and the camera-side waypoints, is pre-taught and stored as a fixed position. When a part is marked as fragile in the configuration, the entire cycle for that part is executed at reduced velocity.

4.7.3 Sequence execution

The sequence executor reads the motion sequence file produced by the planner and runs the cycles one after another. Sequence entries are pick-place pairs identifying the source part and the destination slot, with an optional gripper-width override for non-standard parts. Fragility is read from the configuration file at execution time rather than embedded into the sequence, so that fragility settings can be changed without regenerating the sequence. The executor checks for a cancellation signal between cycles, allowing the operator to abort safely. In that case, the current cycle completes so that the robot returns to Home rather than halting in an arbitrary pose. The number of successfully executed steps is returned to the orchestration layer so that the configuration can be updated to reflect exactly the steps that completed.

4.7.4 Position handling

Positions used during execution come from two sources. Dynamic positions for parts and slots are taken directly from the metric section of the current configuration, which is updated on every vision scan. Static positions, including Home and the camera-side waypoint, are stored in a separate fixed-positions file (the same file used by the teaching tools) and remain constant across sessions. The vision module already produces all metric data in the robot base frame, so no additional coordinate transformation is needed at execution time.

4.7.5 Error handling and recovery

Motion-planning and execution errors propagate as exceptions from MoveIt Commander. The executor catches them, logs the failure, and halts. Typical failure modes are unreachable targets, collision-detected trajectories, and execution drift. The system does not attempt automatic recovery. The cancellation mechanism described above is the only supported way to stop a sequence cleanly.

4.7.6 Coupling to other modules

After execution, the executor returns the number of completed cycles. The configuration module then replays exactly those completed cycles symbolically (Section 4.5), updating part locations and slot occupancy accordingly. An optional automatic post-execution rescan reconciles any discrepancy between the symbolic update and the actual workspace, catching execution errors before they affect the next planning cycle.

4.7.7 Module components

Script Name	Description
Robot_Main.py	Main entry point that initialises ROS, MoveIt, and executes the motion sequence
robot.py	High-level robot interface wrapping MoveIt Commander with motion primitives
sequence_executor.py	Reads motion sequences and drives the robot through each pick-place step
pick_and_place.py	Implements the standardised pick-and-place cycle for part transfers
move_camera_home.py	Utility script to position the robot at the camera home configuration

Table 6: Execution module scripts.

4.8 User interface

The system provides a graphical interface for visual workflow management and interactive configuration. The interface orchestrates the underlying pipeline through a session handler that coordinates module execution and manages state transitions.

4.8.1 GUI architecture

The graphical interface is built with Tkinter, chosen for its minimal dependencies and direct integration with the Python modules used in the rest of the system. The main window is divided into three resizable panels: a chat panel on the left displaying conversational output, an information panel on the upper right showing the current configuration, and a vision panel on the lower right rendering the most recent annotated image from the detection pipeline. A split-view mode in the vision panel supports side-by-side comparison before and after a scene update.

To keep the interface responsive while LLM calls and planning operations run in the background, long-running work is dispatched to worker threads while the main thread is reserved for widget updates. Output produced by the workers and by subprocesses is routed back to the chat panel through a small queue, and the vision image is refreshed automatically whenever the underlying file changes.

4.8.2 Session management

A session handler coordinates the three operational modes. Each mode follows the same pattern: load the relevant configuration (from a fresh scan, the current state, or a memory snapshot), invoke the relevant module (communication, planner, or executor), and persist the resulting state, archiving a timestamped snapshot for later inspection.

4.8.3 Module components

Script Name	Description
Main.py	Application entry point that launches the graphical interface
gui.py	Tkinter-based GUI implementing threaded architecture and message queue system
session_handler.py	Workflow orchestrator managing state transitions and module coordination
run_execute.py	Execution wrapper for robot motion subprocess with ROS environment setup

Table 7: User interface scripts.

4.9 System infrastructure

A small core module provides shared infrastructure used across the rest of the system. Deployment-specific parameters (API keys, model identifier, MoveIt planning groups, gripper joint names, vision thresholds, and filesystem paths) are externalised through environment variables loaded from a `.env` file. This makes it easier to adapt the codebase to other laboratory setups, although porting still requires modifications outside the `.env` file (for example to teach new fixed positions or to adjust hardware-specific calibration). All filesystem locations are imported from a single `paths` module rather than constructed inline. State writes (configuration updates, motion sequences, memory snapshots) use atomic file operations: content is written to a temporary file and then renamed, so readers always observe either the complete old or the complete new contents. Configuration snapshots are archived with a timestamp and a label identifying the operation that produced them, supporting debugging and experiment reproducibility.

4.9.1 Module components

Script Name	Description
<code>config.py</code>	Settings management loading parameters from environment variables
<code>paths.py</code>	Centralised path definitions and atomic file write operations
<code>io_helpers.py</code>	Convenience wrappers for configuration file operations

Table 8: System infrastructure scripts.

5. Results and Discussion

This chapter presents the results of the experimental evaluation described in Chapter 3, organised into four scenarios of increasing complexity. Scenario 1 establishes an unambiguous correctness baseline, Scenario 2 evaluates state-description ambiguity during workspace updates, Scenario 3 evaluates task-specification ambiguity during semantic reconfiguration, and Scenario 4 exercises the complete system across a full end-to-end reconfiguration cycle. Results address configuration correctness (RQ1), ambiguity detection and resolution (RQ2), and the structural drivers of dialogue cost across complexity (RQ3). Complete raw and annotated conversation logs are available in the thesis repository [48]. The summaries drawn from these form the basis of the discussion that follows. The implications of having the system developer act as the primary evaluator are discussed in Section 6.1; throughout this chapter the findings are reported as descriptive evidence from a controlled small-sample evaluation.

5.1 Scenario 1 — Baseline attribute configuration

Scenario 1 uses a fixed workspace consisting of three containers, three kits, and parts in all three colours (Figure 6). This arrangement provides coverage of every supported high-level attribute (see Table 1) within a single consistent setup. All instructions are phrased directly with no deliberate ambiguity, establishing a correctness baseline and minimum turn structure before ambiguity and workspace variation are introduced in subsequent scenarios.

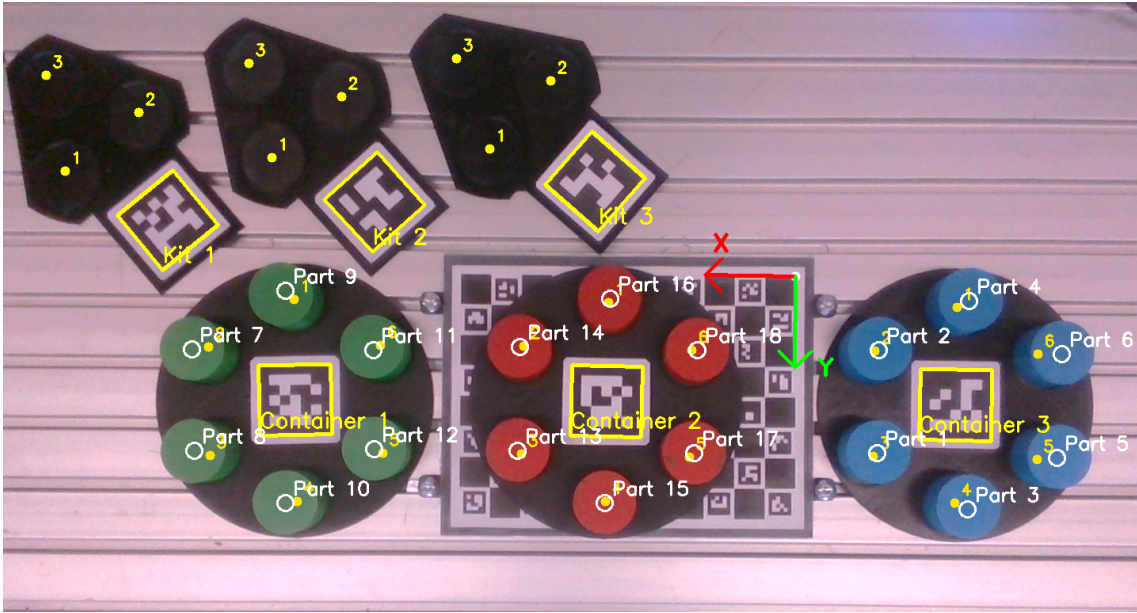


Figure 6: Experimental setup scenario 1.

5.1.1 Results

Table 9 summarises the outcome of all seven trials and their constituent reconfiguration sessions. Each trial targets a distinct high-level attribute. The columns reported throughout this chapter for ambiguity-trial tables are: targeted attribute or ambiguity profile, total turn count, clarification or extra-turn count, and whether the resulting configuration was correct.

Two distinct notions of correctness are reported throughout this chapter. *Final-correct* (column **Correct**) records whether the committed state matches the intended target, regardless of how it was reached. *Autonomous-correct* (column **Auton.**) records whether that correct state was reached without the operator volunteering a parameter the system never requested, correcting a system error, or restructuring an instruction to escape a handler loop. Answering a clarification question the system itself raised does not count against autonomy. Only unprompted operator intervention does. A trial that is final-correct but not autonomous-correct is termed *operator-assisted*, and indicates that final correctness depended on the combined effort of system and operator rather than the system alone.

Trial/Session	Attribute	Turns	Clarif.	Correct
T1-S1	Operational mode	6	0	Y
T1-S2	Operational mode	6	0	Y
T2-S1	Object role	6	0	Y
T2-S2	Object role	6	0	Y
T2-S3	Object role	6	0	Y
T3-S1	Kit recipe	6	0	Y
T3-S2	Kit recipe	6	0	Y
T3-S3	Kit recipe	6	0	Y
T4-S1	Priority	6	0	Y
T4-S2	Priority	8	1	Y
T4-S3	Priority	8	0	Y
T4-S4	Priority	6	0	Y
T4-S5	Priority	6	0	Y
T5-S1	Part compatibility	6	0	Y
T5-S2	Part compatibility	6	0	Y
T5-S3	Part compatibility	6	0	Y
T6-S1	Batch size	6	0	Y
T6-S2	Batch size	6	0	Y
T7-S1	Part fragility	6	0	Y
T7-S2	Part fragility	6	0	Y
Overall				20/20

Table 9: Scenario 1 — Baseline reconfiguration results.

5.1.2 Discussion

All 20 sessions in 7 trials produced correct configurations (RQ1). The dominant pattern was a fixed four-turn structure (opening prompt, user instruction, system proposal, user confirmation), resolving each attribute change in a single exchange and confirming that the proposal-before-commit design adds minimal friction under ideal conditions. This held even for multi-object instructions such as assigning roles to two containers simultaneously or combining compatibility rules across colours in one utterance: the system interpreted and proposed all implied changes together rather than decomposing them.

The single deviation was T4-S2, where “do part 8 first, then part 13” triggered a clarification (“What should be done with Part_8 and Part_13? Should they be moved to a specific container, or prioritised for sorting?”) rather than an immediate proposal, because the system did not directly link “do X first, then Y” to a priority ordering. Once the operator confirmed priority intent, the correct configuration followed in one extra turn. Although incidental rather than designed, this shows the ambiguity detection mechanism remains active even under baseline conditions and that colloquial priority phrasing is not always resolved without clarification. The four-turn minimum established here is the reference structure against which later scenarios are compared.

5.2 Scenario 2 — Update workspace configuration

Scenario 2 evaluates the system’s ability to maintain configuration consistency when the physical workspace is altered between sessions. The experiment uses the update scene sub-mode, in which the system performs a fresh vision scan and must reconcile newly detected parts with the stored configuration from the previous session. The workspace contains a single container with parts in multiple colours, including green parts that share identical visual appearance. One green part carries a fragility attribute from the prior session, making correct identity preservation semantically consequential. The physical changes introduced between sessions consist of one part being removed and another being moved to a new slot, creating a scenario where automatic proximity-based matching alone is insufficient and LLM-mediated identity resolution is required. The scenario comprises six ambiguity trials with progressively larger ambiguity profiles, one volunteer trial, and a constraint violation trial. All ambiguity trials target the same intended mapping outcome, with phrasing varied across trials to cover different combinations of ambiguity types.

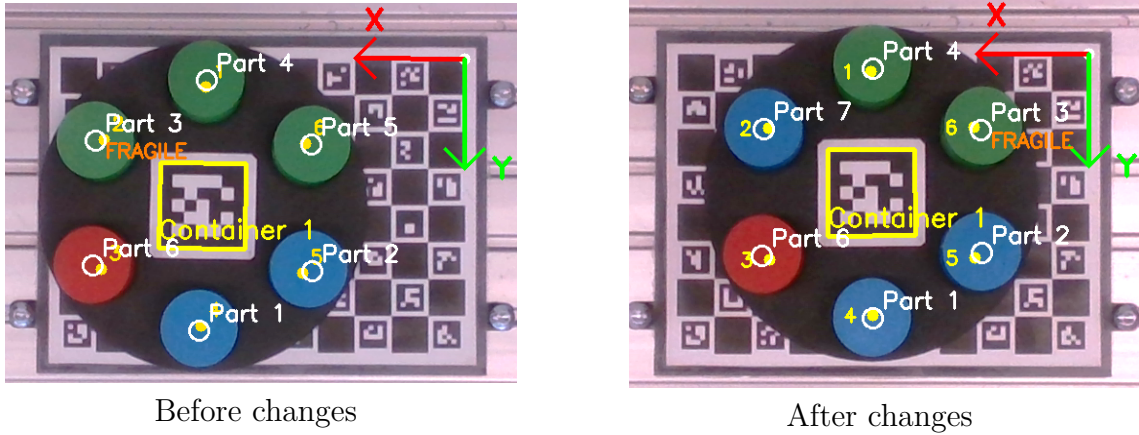


Figure 7: Experimental setup scenario 2.

5.2.1 Results

Table 10 summarises the ambiguity handling trial results, reporting for each trial the ambiguity profile, total turn count, number of extra turns introduced by clarification, and configuration correctness. Table 11 presents the constraint violation trial results, reporting for each sub-trial the violation type, total turn count, and whether the violation was detected by the system.

Trial	Ambiguity Profile	Turns	Extra Turns	Correct	Auton.
T1	None	5	0	Y	Y
T2	R + O	9	2	Y	Y
T3	I + S	6	0	Y	Y
T4	O + S	6	0	Y	Y
T5	R + I + O	9	2	Y	Y
T6	R + I + O + S	9	2	Y	Y
Volunteer	R	8	2	Y	Y
Overall				7/7	7/7

Table 10: Scenario 2 — Workspace update ambiguity results.

CV	Description	Turns	Detected
CV-1	Wrong removed-part count	3	Y
CV-2	Wrong added-part count	3	Y
CV-3	Wrong part moved	3	Y
CV-4	Wrong part removed	3	Y
CV-5	Contradictory scene claim	9	N
CV-6	Non-existent container / wrong colours / off-topic	11	Y
Overall			5/6

Table 11: Scenario 2 — Constraint violation results.

5.2.2 Discussion

All ambiguity trials produced correct configurations (RQ1), and all seven were autonomous-correct: every correct outcome was reached either directly or through clarification questions the system itself raised, with no unprompted operator intervention required. The no-ambiguity baseline was five turns, slightly below the Scenario 1 baseline because the update scene dialogue does not require a separate confirmation step per attribute, indicating the sub-mode adds minimal overhead under unambiguous conditions.

The primary driver of additional turns was referential ambiguity (R), present in T2, T5, T6, and the volunteer trial, each requiring exactly two extra turns: the system issued a clarification before proposing a mapping, correctly identifying that multiple green parts matched the description. This held regardless of co-occurring types, with T6 (R+I+O+S) producing the same overhead as T2 (R+O). I and S were resolved without clarification wherever they appeared without R; in T3 (I+S), “the green part on the right hand side should actually be part 3 from the previous config” carries both an implicit identity claim and a perspective-dependent reference, yet the system proposed the correct mapping in a single turn. The volunteer trial used two instruction turns (first asking to display the scene before correcting the mapping), producing two extra turns despite only a single R being present, and illustrates that

naive users may add turns through exploratory behaviour. The outcome was still correct.

For constraint violations (RQ2), detection was reliable for straightforward factual contradictions between operator claims and scan evidence (CV-1 through CV-4), each resolved in three turns. The undetected case, CV-5, involved a self-contradictory claim (no part removed, yet a part moved to a new slot, physically impossible given the scan). The system did not flag it but resolved it through clarification across several turns, reflecting that logical contradictions internal to the operator’s statements are not explicitly detected. CV-6 showed the system handling a sequence of diverse invalid inputs (non-existent container, colour misidentification, off-topic message) by responding to each in turn and remaining on task.

5.3 Scenario 3, Experiment 1 — Task specification reconfiguration (sorting)

Experiment 1 targets task-specification ambiguity in a sorting context. The workspace contains two containers and parts in three colours (red, blue, and green) with two red parts carrying a fragility attribute. The intended configuration across all trials is identical: sorting mode, red and blue parts routed to Container 1, green parts to Container 2, all containers set as output, green parts prioritised, and red parts marked as fragile. This fixed target configuration allows direct comparison across trials where only the phrasing of the operator’s instruction varies. Six ambiguity trials cover all combinations of the four ambiguity types, progressing from no ambiguity to the full four-type profile, alongside a constraint violation trial. The workspace setup is shown in Figure 8.

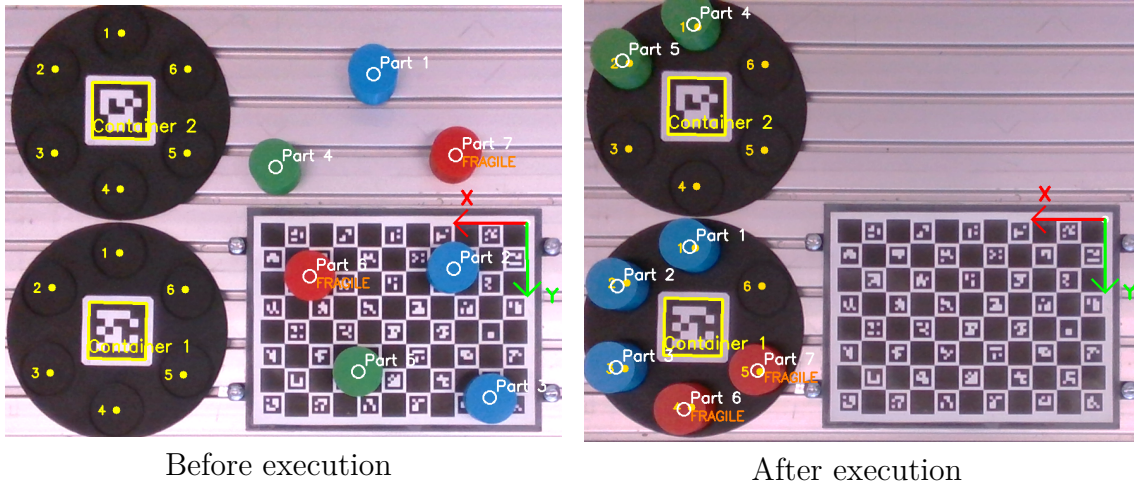


Figure 8: Experimental setup scenario 3 experiment 1.

5.3.1 Results

Table 12 summarises the ambiguity handling trial results, reporting the ambiguity profile, total turn count, extra turns, and configuration correctness for each trial. Table 13 presents the constraint violation results, reporting violation type, turn count, and detection outcome.

Trial	Ambiguity Profile	Turns	Extra Turns	Correct	Auton.
T1	None	6	0	Y	Y
T2	R + O	14	4	Y	N
T3	I + S	6	0	Y	Y
T4	O + S	10	2	Y	Y
T5	R + I + O	9	2	Y	Y
T6	R + I + O + S	14	4	Y	Y
Volunteer	I	12	2	Y	Y
Overall				7/7	6/7

Table 12: Scenario 3, Experiment 1 — Sorting task ambiguity results.

CV	Description	Turns	Detected
CV-1	Non-existent part colour	3	Y
CV-2	Conflicting colour and fragility rules	2	N
CV-3	Capacity exceeded	4	Y
CV-4	Non-existent container	2	Y
CV-5	Conflicting priority	2	Y
Overall			4/5

Table 13: Scenario 3, Experiment 1 — Sorting task constraint violation results.

5.3.2 Discussion

All six ambiguity trials produced correct configurations (RQ1), but only five were autonomous-correct: T2 reached its correct configuration only because the operator volunteered a missing parameter the system had not requested (detailed below). The no-ambiguity baseline was six turns, matching the kitting baseline, consistent with the proposal-before-commit structure established in Scenario 1. The instruction was fully specified in a single turn, confirming that sorting does not carry an inherently higher configuration cost than kitting.

Turn overhead varied considerably. T3 (I+S) required no extra turns: “the red parts are delicate” was interpreted as a fragility assignment without clarification, and the spatial references “bottom”/“top” were resolved against detected container positions via a “describe_scene” tool call, showing that scene context plus the tool is sufficient to ground perspective-dependent references and implicit mappings unaided. T4 (O

without R) required one clarification (two extra turns) for the unspecified green-part destination, while T2 and T6 (R alongside O) each produced four extra turns. In T6 (all four types), a capacity violation was additionally triggered mid-resolution when routing all parts to one container would exceed its slots, prompting the operator to specify full routing before a valid proposal. A notable pattern in T2 was the operator using a confirmation turn to introduce a missing parameter (“green parts need to be sorted into container 2”) rather than simply confirming. The system updated the proposal and re-requested confirmation. While this shows robustness to non-standard confirmation behaviour, it also means T2 is operator-assisted: the system had proposed a configuration omitting the green-part destination and did not flag the gap, so the correct outcome depended on the operator noticing and supplying it.

For constraint violations (RQ2), four of five were detected. The undetected CV-2 involved a logical conflict between two simultaneously specified rules: red parts routed to Container 1 while fragile parts routed to Container 2, with red parts marked fragile, making both applicable to the same objects. The system accepted both without flagging the conflict, whereas direct factual violations (non-existent colours, exceeded capacity, missing containers, conflicting priorities) were caught consistently. The volunteer trial produced a correct configuration and is notable for two reasons: the volunteer spread the configuration across three instruction turns rather than one compound instruction, raising the turn count despite a simple ambiguity profile. The instruction “I want to sort the colours in their respective container” was classified as I, implying sorting mode and output roles without stating them, which the system identified and confirmed before applying. A typo (“con-taier”) was handled without disruption, suggesting robustness to minor input noise.

5.4 Scenario 3, Experiment 2 — Task specification reconfiguration (kitting)

Experiment 2 targets task specification ambiguity in a kitting context. The workspace contains three containers as part sources and three kits as assembly targets, with parts available in all three colours. The intended configuration across all trials is: kitting mode, Container 1 and Container 2 as input, all three kits as output, a batch size of two, and a kit recipe of one red and one blue part per kit. As in Experiment 1, this fixed target configuration allows direct comparison across trials where only the operator’s phrasing varies. Six ambiguity trials cover all combinations of the four ambiguity types, alongside a constraint violation trial. The workspace setup is shown in Figure 9.

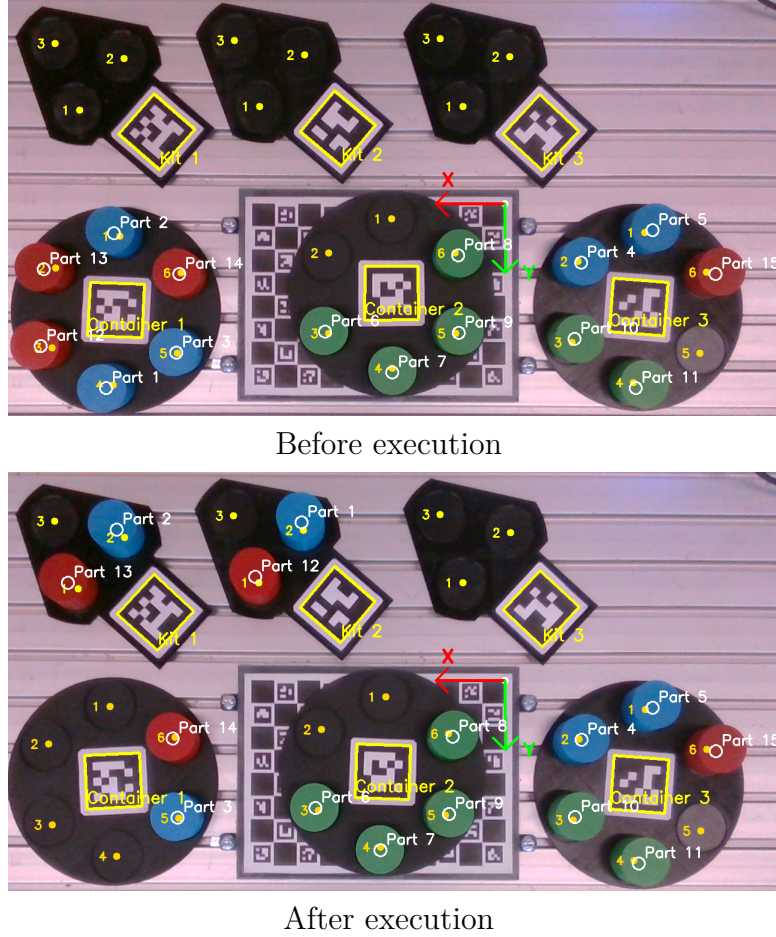


Figure 9: Experimental setup scenario 3 experiment 2.

5.4.1 Results

Table 14 summarises the ambiguity handling results, reporting the ambiguity profile, turn count, extra turns, and correctness per trial. Table 15 presents the constraint violation results with violation type, turn count, and detection outcome.

Trial	Ambiguity Profile	Turns	Extra Turns	Correct	Auton.
T1	None	6	0	Y	Y
T2	R + O	14	4	Y	Y
T3	I + S	12	4	Y	N
T4	O + S	18	6	Y	N
T5	R + I + O	12	4	Y	Y
T6	R + I + O + S	14	6	Y	Y
Volunteer	None	8	0	Y	N
Overall				7/7	4/7

Table 14: Scenario 3, Experiment 2 — Kitting task ambiguity results.

CV	Description	Turns	Detected
CV-1	Kit recipe exceeds kit capacity	2	Y
CV-2	Batch size exceeds kit count	3	Y
CV-3	Insufficient parts for recipe	2	Y
Overall			3/3

Table 15: Scenario 3, Experiment 2 — Kitting task constraint violation results.

5.4.2 Discussion

All seven ambiguity trials produced correct configurations (RQ1), but only four were autonomous-correct. In the remaining three (T3, T4, and the volunteer trial) the correct outcome depended on the operator intervening beyond answering the system’s own questions. The no-ambiguity baseline was six turns, as T1 was expressed as a single compact instruction covering all required attributes and resolved in one proposal cycle.

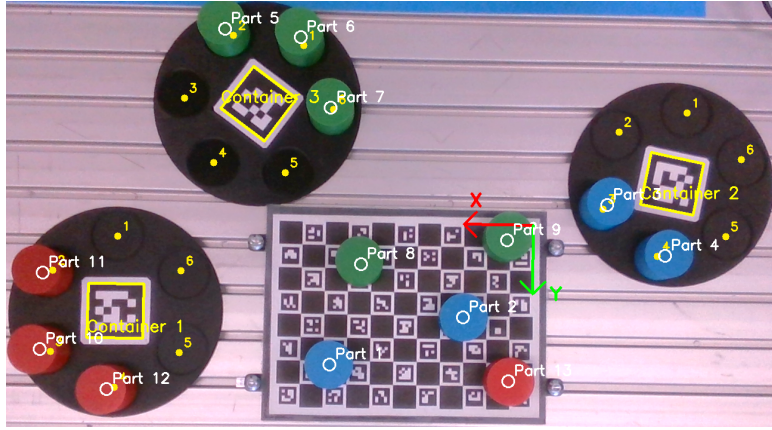
An important qualification applies. Correctness is measured as a binary outcome (committed state matches target), and by this metric every trial succeeded. However, in three trials the correct outcome was reached only because the operator proactively supplied missing parameters the system did not flag, or corrected a system error. T4 (O+S) is the clearest example: after the first proposal the system was prepared to commit a configuration containing only operational mode, batch size, and one container exclusion, without having asked about container roles, kit assignments, or the recipe. The operator intervened (“yes, but what about the other parameters needed for kitting”), after which the system asked which parameters to

set but still required the operator to enumerate them. T3 (I+S) followed the same pattern on a smaller scale: the first proposal omitted container and kit roles entirely, and the operator had to volunteer “I need object roles for kitting” before the system addressed them. The volunteer trial was operator-assisted for a different reason: the instruction was unambiguous, but the system raised a false constraint violation (computing parts for all three kits rather than respecting the batch size of two), and the volunteer had to correct this reasoning twice before the system accepted the instruction. In all three cases the system never proactively audited the proposal for completeness against the requirements of kitting mode, so correctness reflects the combined capability of system and operator rather than autonomous omission detection.

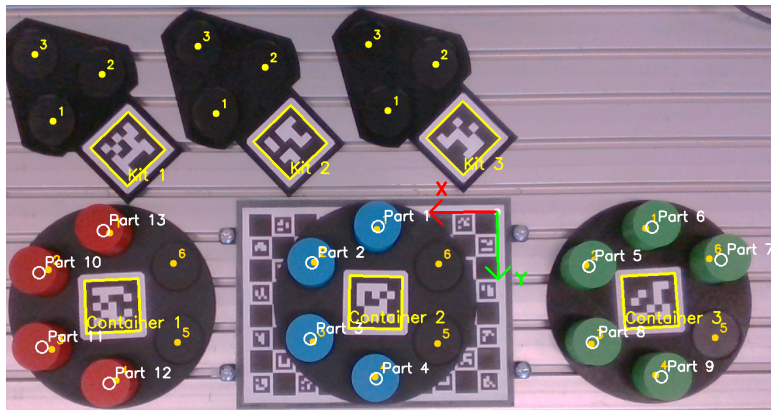
O was the dominant overhead driver. Kitting requires more interdependent parameters than sorting (mode, roles, recipe, batch size must all be consistent), so omissions cascade: trials with O alongside other types introduced four to six extra turns depending on how many parameters were missing, with T4 producing the highest count of the experiment at eighteen turns. A notable pattern in T5 (R+I+O) was the operator delegating which two kits to fill (“you decide”, “doesn’t matter, just need to be 2 kits”). After two unsuccessful elicitation attempts, the system resolved this autonomously by selecting Kit 1 and Kit 2, a deliberate default-when-indifferent behaviour that recurred consistently in T6. For constraint violations (RQ2), all three were detected, each a quantitative check against the workspace state (recipe exceeding capacity, batch size exceeding kit count, insufficient parts), flagged immediately. The volunteer trial used an unambiguous single-turn instruction matching T1 and was correct, but involved a false violation: the system computed that three blue parts were needed, reasoning over all three kits rather than respecting the stated batch size of two, requiring two corrections before accepting the instruction. This is a system-side constraint-checking error, not user ambiguity, and highlights a failure mode where batch size is not factored into parts-availability validation.

5.5 Scenario 4 — Full end-to-end reconfiguration cycle

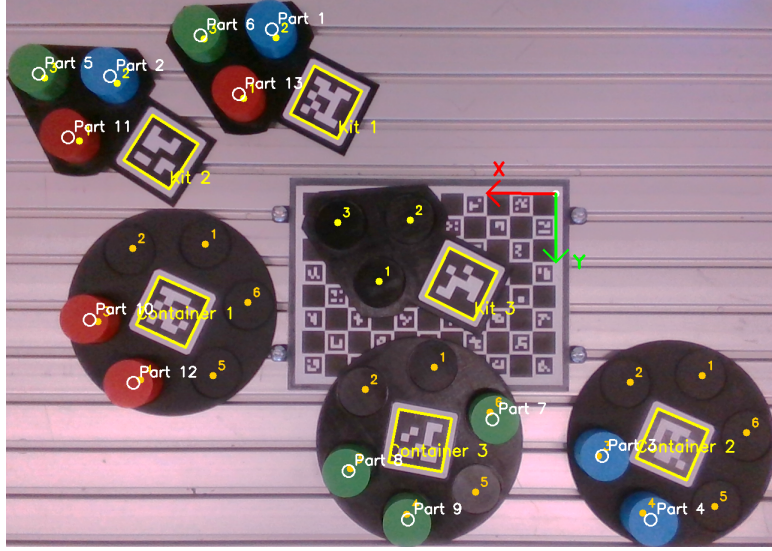
Scenario 4 evaluates the complete integrated system across a sequential four-phase reconfiguration cycle under conditions representative of a real high-mix, low-volume production changeover. All system components are exercised in sequence: vision scanning, LLM-mediated configuration dialogue, PDDL planning, and robot execution. The scenario is performed as a single trial, as the goal is to assess end-to-end system coherence rather than ambiguity coverage. The four phases progress as follows: Phase 1 establishes an initial sorting configuration from a fresh scan and executes the resulting motion sequence. Phase 2 introduces kits, performs an update scene cycle to reconcile the new physical state, and configures a kitting task. Phase 3 modifies the kit recipe and fills the remaining empty kit without a new scan. Phase 4 refills the containers and updates the kit recipe to include fragile parts. The first three phases are shown in Figure 10.



Phase 1



Phase 2



Phase 3

Figure 10: Experimental setup scenario 4.

5.5.1 Results

Table 16 summarises the results across all four phases of the end-to-end cycle, reporting for each phase the description, total turn count, extra turns introduced by clarification or constraint handling, and configuration correctness.

Phase	Description	Turns	Extra Turns	Correct	Auton.
1	Sorting config + execution	6	0	Y	Y
2	Scene update + kitting config	14	2	Y	Y
3	Kit recipe change	18	8	Y	N
4	Scene update + fragile recipe	28	8	Y	Y
Total		66	18	4/4	3/4

Table 16: Scenario 4 — Full end-to-end cycle results by phase.

5.5.2 Discussion

All four phases completed with correct configurations (RQ1). Three were autonomous-correct. Phase 3 was operator-assisted: final correctness depended on the operator manually restructuring the instruction to escape a constraint-violation loop the system could not resolve on its own (detailed below). The total of 66 turns reflects the cumulative cost of a multi-phase reconfiguration involving two vision rescans, two robot executions, and four configuration dialogues.

Phase 1 completed with no extra turns, resolving the full sorting configuration in a single instruction and proposal cycle. Phase 2 introduced two extra turns from a

single clarification over which two of three kits to fill. The update scene dialogue required no identity resolution, as the new kits were detected as new objects and accepted without correction, and the operator’s confirmation turn additionally cleared prior priorities and compatibility rules, which the system handled before committing. Phase 3 produced the highest count at 18 turns and eight extra turns, driven by a recurring constraint-violation loop: the operator instructed the system to change the recipe and fill the remaining empty kit, but Kit 1 and Kit 3 were already full, making the instruction physically impossible. The system correctly detected the capacity constraint but re-raised the same warning across four consecutive turns rather than resolving after the first response, requiring the operator to explicitly restructure the instruction (removing output roles from the filled kits and deleting the batch size) before proceeding. The final configuration was correct. Phase 4 was the most complex dialogue at 28 turns and eight extra turns, covering a full workspace replacement and a fragility-based recipe. The update scene accepted all automatic matches without correction (all parts replaced, no ambiguous identities), and the configuration required four clarification turns to resolve the recipe: “1 fragile part per kit” prompted the system to clarify whether fragility was a selector for currently fragile parts or a general future-applicable rule, and whether the non-fragile recipe parts should also be fragile, each resolved in one turn. The operator’s confirmation turn then added Kit 1 and Kit 3 as outputs, triggering a further clarification and proposal cycle before committing.

The Phase 3 loop is the clearest example in this scenario of an architectural cost driver beyond task complexity itself: the constraint-violation handler re-issues the same warning each turn without retaining the operator’s prior responses, forcing manual restructuring to break out. The detection is functionally correct, but the recovery dialogue around it contributes overhead not inherent to the task. This and the broader turn-cost patterns are analysed across scenarios in Section 5.6.3 below.

5.6 Cross-scenario analysis

This section synthesises the experimental findings across all four scenarios, addressing all three research questions from a system-wide perspective. Where RQ1 and RQ2 are concerned, the relevant data is drawn from the per-trial results reported in the scenario tables above, enabling direct comparison of the same ambiguity profiles and violation types across different task domains. For RQ3, the analysis draws on the aggregated turn-type distributions reported in Table 17, which consolidates all trials (including constraint violation sub-trials) across all scenarios into a single comparative overview.

Metric	S1	S2	S3 E1	S3 E2	S4
Baseline turns	6	5	6	6	6
Max turns	8	9	14	18	28
Extra turns	0–1	0–2	0–4	0–6	0–8
USR-INST	20	16	13	12	8
USR-CONF	21	7	11	9	14
USR-CLAR	1	5	6	14	3
AST-PROP	21	9	15	14	11
AST-CLAR	1	5	5	14	5
AST-VIOL	0	6	5	3	4
Correctness	20/20	7/7	7/7	7/7	4/4
Auton.-correct	20/20	7/7	6/7	4/7	3/4
CV detection	n/a	5/6	4/5	3/3	n/a

Table 17: Cross-scenario turn-type summary (all trials including CV sub-trials).

5.6.1 RQ1: configuration correctness across complexity

Across all scenarios and phases, the system produced correct final configurations in all 45 evaluated configuration sessions. Of these, 40 were autonomous-correct: reached without the operator volunteering an unrequested parameter, correcting a system error, or restructuring an instruction to escape a handler loop. The remaining five were operator-assisted (one in Scenario 3 Experiment 1, three in Scenario 3 Experiment 2, one in Scenario 4, and none in Scenarios 1 and 2), and are concentrated in the kitting domain, which carries the highest number of interdependent required parameters. In a separate set of 14 constraint violation sub-trials, 12 violations were detected (the two undetected cases are discussed under RQ2). Taken together, this constitutes strong evidence of feasibility: the system reliably translates natural language instructions into correct executable configurations across a range of conditions, from simple single-attribute changes to full multi-phase production changeovers.

However, configuration correctness must be interpreted carefully. The correctness metric records whether the final committed state matches the intended target, and by this measure every session succeeded. It does not record whether the system autonomously identified all missing or ambiguous parameters. In several tri-

als, most visibly Scenario 3 Experiment 2 Trial 4, the operator had to proactively identify parameters that the system was prepared to commit without. In T4, the system proposed a configuration covering only operational mode, batch size, and one container exclusion. The operator intervened with “yes, but what about the other parameters needed for kitting” before the system addressed container roles, kit assignments, or the kit recipe. The correct outcome was reached, but through operator-driven recovery rather than system-initiated completeness checking. This pattern, where the operator compensates for the system’s lack of parameter enumeration, appeared in other trials as well and means that the 100% correctness rate reflects the combined capability of system and operator rather than system-autonomous performance alone.

This distinction has a direct architectural implication: the system does not maintain an explicit schema of required parameters per operational mode and therefore cannot audit a proposal for completeness before presenting it to the operator. The kitting domain is particularly exposed to this gap because it has the highest number of interdependent required parameters of any tested configuration domain. The proposal-before-commit mechanism remains a reliable final safeguard, but only when the operator actively audits the proposal, which may not be reliable in practice if proposals are long or the operator is fatigued.

Scenario 4 confirmed correctness across all four phases of the full end-to-end cycle. This is significant because Phases 3 and 4 both accumulated substantial extra turns due to constraint handling and recipe underspecification respectively, yet neither produced a configuration error, demonstrating that the system maintains state coherence across extended multi-phase sessions even when dialogue cost is high.

5.6.2 RQ2: ambiguity detection and resolution

The most informative cross-scenario comparison for RQ2 is to examine how the same ambiguity profiles produce different turn costs across different task domains. Tables 9, 11, and 13 allow this directly, since Scenarios 2, 3 Experiment 1, and 3 Experiment 2 all test the same four ambiguity types (R, I, O, S) in isolation and combination, applied to increasingly complex configuration domains.

Under no ambiguity, the baseline turn count is consistent across task domains: five turns in Scenario 2 (workspace update) and six turns in both Scenario 3 Experiment 1 (sorting) and Experiment 2 (kitting). This confirms that the system’s minimum interaction cost is determined by the proposal-before-commit structure rather than by domain-specific attribute complexity.

Referential ambiguity (R) is the most stable type across domains. In Scenario 2, every trial containing R incurred exactly two extra turns regardless of co-occurring ambiguity types: T2 (R+O), T5 (R+I+O), and T6 (R+I+O+S) all produced the same overhead. This stability arises because referential ambiguity triggers a targeted, well-scoped clarification: the system identifies that multiple objects satisfy the description, names the candidates, and asks the operator to specify. The operator’s single response resolves it. This pattern recurs consistently in Scenario 3 Experiment 1, where R-containing trials similarly converge around the same overhead ceiling regardless of additional types. The behaviour reflects a genuine strength: be-

cause the system has direct access to the detected object identities from the vision module, it can precisely identify when a referent is underspecified and ask exactly the right question.

The cost of implicit (I) and spatial (S) ambiguity is more domain-sensitive. In Scenario 2 and Scenario 3 Experiment 1, I+S required zero extra turns in both cases, since the system resolved spatial references via the scene description tool and inferred implicit constraints from context without asking. In Scenario 3 Experiment 2, the same profile (I+S) cost four extra turns. Reading Trial 3 of that experiment, the implicit constraint in question was a role exclusion (“do not use the container on the right hand side”), and the spatial reference triggered a clarification because three containers share the workspace, making the perspective-dependent reference genuinely underresolved. The additional turns then emerged because the proposal that followed omitted container roles, requiring a second instruction turn from the operator. This illustrates that I+S ambiguity does not carry a fixed cost: its resolution depends on how much context the current scene provides. In simpler workspaces with fewer objects, spatial and implicit references are resolved contextually; in denser workspaces with more objects and roles to assign, the same phrasing generates genuine ambiguity that the system cannot resolve without asking.

Omission (O) is the dominant cost driver in the kitting domain and the clearest differentiator between Scenario 3 Experiment 1 and Experiment 2. Comparing O+S across experiments: Experiment 1 produced two extra turns, Experiment 2 produced six. Comparing R+I+O+S: Experiment 1 produced four extra turns, Experiment 2 produced six. The kitting domain requires more interdependent parameters (mode, container roles, kit roles, recipe, and batch size), meaning that a single omitted parameter can cascade into multiple clarification cycles as each recovered parameter exposes the next gap. However, this characterisation requires qualification: the system pursues missing parameters when they are syntactically obvious (such as an incomplete sentence) or when the operator explicitly draws attention to them, but it does not proactively enumerate required parameters against a schema. In Scenario 3 Experiment 2 Trial 4, the system would have committed a configuration missing container roles, kit assignments, and the kit recipe had the operator not intervened after the first proposal. The omission handling mechanism is therefore reactive rather than proactive, and its effectiveness in complex domains depends substantially on operator vigilance. The non-linear scaling of omission cost reflects both the structural complexity of the kitting configuration space and the increasing burden placed on the operator to compensate for the system’s lack of completeness checking.

Scenario 3 Experiment 2 also revealed a deliberate design behaviour under repeated operator delegation: in Trials 5 and 6, the operator responded to the system’s clarification about which kits to fill with “you decide” and “doesn’t matter, just need to be 2 kits.” After two unsuccessful elicitation attempts, the system resolved the ambiguity autonomously by selecting Kit 1 and Kit 2, producing a correct configuration. This autonomous fall-through (making a reasonable default when the operator signals indifference) is a meaningful interaction property. It prevents the clarification loop from continuing indefinitely when the operator genuinely has no preference, and it occurred consistently across both trials that triggered it.

Constraint violation detection showed a consistent two-tier pattern. Factual violations against the current workspace state (wrong part counts, non-existent containers, exceeded capacities, explicit priority conflicts) were detected reliably and resolved within two to three turns across all scenarios. The undetected violations in Scenario 2 (CV-5) and Scenario 3 Experiment 1 (CV-2) both involved logical conflicts that required cross-statement or cross-attribute reasoning rather than direct comparison against scan evidence. In CV-5, the operator’s description was internally self-contradictory. In CV-2, a fragility rule and a routing rule jointly implied an impossible assignment only when evaluated together. In both cases the system accepted the input without flagging the conflict. The constraint-checking mechanism therefore has a clear structural boundary: it enforces factual consistency against the scan reliably, but does not reason over the combined logical implications of simultaneously specified rules.

Scenario 4 Phase 4 provided the strongest positive evidence for LLM-based ambiguity resolution specifically. When the operator specified “1 fragile part per kit,” the system asked two semantically precise clarification questions: whether fragility should be tied to currently marked parts or treated as a general future-applicable rule, and whether the non-fragile parts in the recipe should also be fragile. Both questions identified genuine semantic ambiguities in the instruction that a keyword-based or rule-based system would have silently resolved via a fixed default. The operator’s responses produced a richer and more intentional configuration than any non-LLM approach would have yielded.

5.6.3 RQ3: structural drivers of dialogue cost

Table 17 reveals a coherent pattern across all scenarios that identifies the structural drivers of dialogue cost and how they scale with task complexity and ambiguity type. The analysis is internal to the system: it compares the dialogue against itself across scenarios rather than against an external baseline such as expert reprogramming or a structured GUI. The findings should therefore be read as a characterisation of where dialogue cost originates within this system, not as absolute efficiency claims.

The first observation is that baseline dialogue cost is consistent across task domains. Under unambiguous conditions, both sorting (Scenario 3 Experiment 1) and kitting (Scenario 3 Experiment 2) resolved in six turns, and workspace update (Scenario 2) in five turns. The small difference between five and six turns reflects the update sub-mode’s simpler confirmation flow rather than domain attribute complexity. Baseline cost is therefore determined primarily by the proposal-before-commit structure, not by the number of attributes the operational mode requires.

The second observation concerns how ambiguity types add cost. Referential ambiguity adds a fixed two-turn overhead regardless of co-occurring types, because each instance triggers a single targeted clarification that the operator resolves in a single reply. Implicit and spatial ambiguity often add zero overhead in workspaces with sparse or clearly arranged objects, because the system resolves them through context, but they can require clarification in dense workspaces where the same phrasing becomes genuinely under-resolvable. Omission is the most variable: in simple domains a missing parameter triggers a single clarification, but in complex domains

with many interdependent parameters, each recovered parameter exposes the next gap, producing a cascading clarification chain. The cost of omission therefore scales non-linearly with the number of required parameters in the task domain, while the cost of referential ambiguity remains constant. This distinction is what produces the large gap in AST-CLAR counts between Scenario 3 Experiment 1 (five clarifications across the experiment) and Scenario 3 Experiment 2 (fourteen clarifications across the experiment) even though both experiments share the same ambiguity profiles.

The third observation concerns the proposal-confirmation structure. In Scenario 1, AST-PROP and USR-CONF are equal at 21: every proposal received exactly one confirmation, with no revisions or rejections. This strict one-to-one symmetry establishes the clean baseline of the proposal-before-commit design. As complexity increases the symmetry blurs, but in two different ways that carry different cost implications. In Scenarios 2 and 3 Experiment 1 the asymmetry comes from proposals that required revision before confirmation; this is a moderate, expected cost of dialogue refinement. In Scenario 4 USR-CONF actually exceeds AST-PROP, because the operator frequently combined confirmation with new instructions in the same turn, which the system handled without adding turns. The compound-turn pattern is therefore not a cost driver: it absorbs operator input efficiently rather than fragmenting it across additional turns.

The fourth observation is that two specific architectural patterns produce dialogue cost beyond what task complexity itself demands, and these are the primary findings of RQ3. The first is the absence of proactive parameter completeness checking. Because the system does not maintain an explicit schema of required parameters per operational mode, it cannot audit a proposal for completeness before presenting it to the operator. In the kitting domain this caused omission cost to scale non-linearly, as the operator had to identify and supply missing attributes through additional instruction turns. The interaction in Scenario 3 Experiment 2 most closely resembles a structured interview, with USR-CLAR equal to AST-CLAR at 14 and the operator’s role shifting from instruction-giver to question-answerer. Whether this dynamic is acceptable depends on use case, but it represents overhead that a parameter schema could substantially reduce.

The second architectural cost driver is the stateless constraint violation handler. The USR-CLAR column mirrors AST-CLAR exactly in all scenarios except Scenario 4 (5 AST-CLAR, 3 USR-CLAR). This asymmetry is directly visible in the Phase 3 log: the system issued the same constraint-violation warning four consecutive times, and the operator’s attempts to resolve it (“only fill kit 2”, “yes”, “forget about the kit, just change the kit recipe”) were classified as USR-INST turns rather than USR-CLAR responses, because the operator was restructuring the instruction rather than answering the system’s question. Two of the five AST-CLAR turns therefore generated no corresponding clarification response, because the system’s question was no longer the right question after the first iteration. The detection mechanism is functionally correct, but the dialogue around recovery re-evaluates the same state on every turn without retaining memory of prior operator responses, forcing the operator to carry the full burden of breaking the loop.

A judgement of absolute dialogue efficiency lies outside what the data presented here can support. The turn counts and turn-type distributions characterise the in-

ternal cost structure of the dialogue but cannot, on their own, determine whether the resulting interaction is faster, slower, or more burdensome than alternative reconfiguration approaches. Against expert reprogramming the system is plainly faster, because reprogramming requires code changes that the operator cannot perform at all. Against a purpose-built GUI with attribute fields the dialogue would likely be slower for well-defined repetitive changes but more flexible for novel or under-specified ones. Establishing either of these comparisons rigorously would require a separate user study and is left for future work. The structural analysis presented in this section identifies the cost drivers that any such comparison would need to account for, and the two architectural patterns identified above are the primary candidates for reducing dialogue cost beyond what task complexity itself imposes.

5.6.4 Overall system assessment

The cross-scenario results collectively demonstrate that the system achieves its core feasibility goal: an operator can reconfigure a physical robot workspace through natural language dialogue without manual code editing or prior robotics expertise, and the system translates these instructions into correct executable configurations across conditions representative of real high-mix, low-volume production changeovers.

Strengths. The most consistent strength is robustness to phrasing variation: all 45 configuration sessions produced correct final configurations despite deliberate variation across all tested ambiguity profiles, and 40 of these were reached autonomously, without operator intervention beyond answering the system’s own clarification questions. Referential ambiguity detection is reliable and tightly coupled to the vision module’s object identity output, providing a dependable safeguard against the most semantically consequential misassignment errors. Factual constraint checking catches the large majority of violations (12 of 14 sub-trials) before they propagate into committed configurations, consistently within two to three turns. The system’s autonomous fall-through behaviour under repeated operator delegation prevents indefinite clarification loops. Compound confirm-plus-instruct turns are handled correctly and without disruption. Finally, the LLM-based approach enables semantically precise clarification of complex constructs such as fragility-rule specifications, producing richer configurations than keyword-based alternatives would yield.

Limitations. The primary limitation is the absence of proactive parameter completeness checking: the system does not maintain an explicit schema of required parameters per operational mode and therefore cannot audit a proposal for completeness before presenting it to the operator. In complex multi-attribute domains such as kitting, this places the burden of verifying that all required attributes are covered on the operator, who must actively identify gaps that the system does not flag. The 100% configuration correctness rate therefore reflects the combined performance of system and operator and should not be interpreted as evidence that the system autonomously resolves all omission ambiguity. The autonomous-correct rate of 40 of 45 sessions is the more conservative measure of system-alone performance. Cross-attribute logical conflict detection is structurally absent, as the constraint-checking mechanism evaluates each operator claim against the workspace state individually rather than reasoning over combined logical implications, accounting for both unde-

tected constraint violations. Two architectural patterns drive dialogue cost beyond what task complexity itself demands: the absence of proactive parameter completeness checking, which causes omission cost to scale non-linearly in complex domains, and the stateless constraint-violation handler, which forces the operator to manually restructure instructions when a violation cannot be resolved in a single clarification cycle. Both are addressable in future iterations of the system rather than intrinsic to the conversational approach.

6. Conclusion

This thesis investigated how conversational interaction, supported by vision-based perception, can enable non-expert operators to perform semantic task-level reconfiguration of modular robotic manipulation systems in high-mix, low-volume production environments. The core problem targeted was the dependence on robotics specialists for configuration changes that operators could describe in a single natural language sentence, a bottleneck that limits production flexibility and increases reconfiguration overhead precisely in the environments where adaptability is most needed.

To address this, a fully integrated system was designed and implemented comprising five interconnected modules: a vision pipeline for object detection and workspace scanning, a conversational communication module for multi-turn dialogue with ambiguity detection and constraint checking, a configuration module for symbolic world state management across sessions, a PDDL-based planning module for deterministic sequence generation, and an execution module for robot motion on a Franka Emika Panda. The system was evaluated across four experimental scenarios of increasing complexity, from single-attribute baseline configuration to full multi-phase end-to-end production changeovers.

On configuration correctness (RQ1), all 45 sessions across all scenarios produced correct final configurations, 40 of them without operator intervention beyond answering the system’s own clarifications, and 12 of 14 constraint-violation sub-trials were correctly detected, holding under all tested ambiguity profiles and across all four phases of the full cycle. However, correctness in complex domains depended partly on the operator proactively identifying missing parameters the system did not flag: in Scenario 3 Experiment 2 the system was prepared to commit incomplete configurations until the operator intervened. The 100% rate therefore reflects the combined capability of system and operator, with proposal-before-commit serving as a reliable safeguard only when the operator audits the proposal. This reveals a structural limitation: the system maintains no explicit schema of required parameters per mode and cannot proactively check proposal completeness, so deployment would require either proactive completeness checking or operator training to verify proposals before confirmation.

On ambiguity (RQ2), the system showed consistent two-tier behaviour. Referential ambiguity was detected reliably wherever it appeared, producing a targeted clarification resolved in exactly two extra turns regardless of co-occurring types, attributable to the vision module supplying explicit object identity that lets the LLM detect when a description is underspecified relative to the scene. Implicit constraint and spatial ambiguity were most often resolved through contextual inference without operator involvement, though their cost rose substantially in the kitting domain where density and parameter interdependency weaken scene context. Omission ambiguity was the most variable and exposed a detection-versus-resolution gap: correct configurations were ultimately produced, but handling was reactive rather than proactive, pursuing missing parameters only when syntactically obvious or operator-flagged, placing substantial burden on the operator in the kitting domain. Constraint-violation de-

tection was reliable for factual violations against the workspace state (12 of 14) but missed the two cases involving logical conflicts requiring reasoning over the combined effect of simultaneously specified rules.

On dialogue cost (RQ3), baseline cost was consistent across task domains: six turns for both sorting and kitting, and five turns for workspace updates, confirming that the proposal-before-commit structure rather than domain attribute complexity determines the minimum interaction cost. Extra turns from ambiguity scaled predictably with type: referential carried a fixed two-turn overhead, while omission-driven overhead scaled non-linearly with the number of interdependent parameters. Two architectural patterns were the dominant drivers of additional cost beyond task complexity: the absence of proactive completeness checking, which causes omission cost to cascade and shifts the operator from instruction-giver to question-answerer. The stateless constraint-violation handler re-issues the same warning each turn without retaining prior operator responses, forcing manual restructuring to break recovery loops. Both are addressable in future iterations rather than intrinsic to the conversational approach. Absolute dialogue efficiency could not be judged from this data, as no comparative baseline was established. The structural analysis instead identifies the cost drivers any such comparison would need to account for.

Taken together, the findings confirm the thesis aim: a non-expert operator can reconfigure a collaborative robotic pick-and-place system through natural language dialogue, without manual code editing or specialist involvement, across a range of conditions representative of real high-mix, low-volume production changeovers. The system demonstrates feasibility of the integrated architecture and identifies the structural drivers of dialogue cost that would need to be addressed to scale the approach to more complex domains and broader operator populations.

6.1 Critical discussion

The most significant methodological limitation of this work stems from the fact that the system designer and primary evaluator were the same person. Throughout the experiments, interactions with the system followed a consistently efficient and structured pattern, with repeated use of the same vocabulary and compact instruction phrasing that a real operator unfamiliar with the system would not naturally reproduce. This familiarity likely inflated the efficiency metrics relative to what a broader operator population would achieve. The volunteer trials made this gap visible in a concrete way: different phrasings, unfamiliar synonyms, and occasional spelling errors triggered failures that had not been observed during developer-led testing. These specific issues were corrected before final reporting, but it is acknowledged that further testing with a wider participant group would surface additional cases of the same nature. The system as evaluated is therefore more accurately characterised as robust to the developer’s interaction style than to natural operator variation in general.

Two findings during development and testing were not fully anticipated. The first was the extent to which LLM response quality varies across seemingly similar inputs. In some cases a single compact instruction was sufficient to infer all required attributes and produce a complete proposal without clarification. In other cases, an equally simple statement led to unnecessary clarification questions or a missed parameter. This non-determinism is difficult to account for in a controlled evaluation, since the same input on a different run may produce a different dialogue path, and it highlights that LLM-based systems are probabilistically capable rather than guaranteed correct, a distinction with meaningful implications for any deployment in a safety-critical environment. The second unanticipated difficulty concerned the priority system. Attempts to model a multi-dimensional priority structure (where both part colour and container carry independent priorities that can conflict) resulted in a significantly more complex PDDL representation than initially expected. PDDL 3 preference constraints were explored as a means of expressing this more naturally, but the resulting computation times were impractical for an interactive session flow. A single-dimension priority scheme was ultimately retained, which meant accepting that certain priority interactions are not expressible in the current planning representation.

In hindsight, the most impactful architectural change would have been the implementation of an explicit parameter schema per operational mode. As currently designed, the user must be aware of which attributes are required for a given task mode (for example, which parameters must be set in the configuration interface for kitting or sorting to produce a valid PDDL problem) and must verify through the GUI that all relevant attributes are correctly populated before initiating planning. The system itself provides no guidance on this: if an optional parameter is absent, the LLM does not mention it. If a mandatory attribute is missing, the PDDL planner fails silently without producing a message that identifies what is missing or why planning could not proceed. The evaluation confirmed that this limitation has practical consequences: in Scenario 3 Experiment 2, the system was prepared to commit configurations missing required attributes, and the operator had to intervene explicitly to ensure completeness. While the configuration space in this thesis was small enough that an attentive operator could compensate, scaling to more complex task domains with a larger set of attributes would make silent planner failures and incomplete proposals a significant usability problem. A more robust design would have the LLM proactively flag missing required attributes during the configuration dialogue and mention optional parameters the user may have overlooked, and the planning module would produce a clear, human-readable error message identifying the missing attribute when planning fails rather than returning an empty or failed result.

Several other choices made during the project were constrained by available resources and time rather than optimality. The robotic platform was selected in part because a general robot control class had already been developed for it during a prior project, making reuse straightforward. The camera was used because it was already physically mounted on the robot rather than through a systematic sensor evaluation. The experimental parts, kits, and containers were intentionally simplified to keep detection, grasping, and symbolic reasoning tractable within the project timeline.

Real industrial parts introduce varied shapes and surface finishes that would challenge both the vision pipeline and the gripper mechanics, and real storage containers introduce bin-picking complexity that constitutes a significant engineering problem in its own right. The LLM was selected based on academic prevalence and reliability rather than a rigorous benchmark comparison, as access to the most capable available models was constrained by financial and hardware limitations. A different model might require less prompt engineering or handle edge cases differently, meaning the results characterise this specific system under these specific conditions rather than LLM-based conversational configuration in general. These constraints are acknowledged openly: the thesis demonstrates that the proposed architecture is feasible and characterises its failure modes, which is considered a valid research contribution, but the degree of simplification relative to a real industrial deployment should be kept in mind when interpreting the findings.

From a professional development perspective, the project required simultaneous work across disciplines that are rarely combined within a single student project: mechanical design and fabrication, computer vision, symbolic AI planning, multi-module software architecture, and LLM integration. Working with a commercial API, structuring prompt engineering, and debugging probabilistic rather than deterministic failures provided practical exposure to modern software development practices that extend well beyond conventional robotics coursework. The project also established a structured version control workflow that is considered a lasting outcome of the work.

6.2 Generalisation of the result

This section situates the system’s results within a broader context, examining the transferability of the architecture to other domains and its position relative to existing literature.

6.2.1 Applicability to other domains

This thesis was designed specifically for high-mix, low-volume collaborative robotic pick-and-place, where the core challenge is enabling non-expert operators to express semantic task changes quickly and reliably during frequent production changeovers. However, the underlying architecture (vision-grounded conversational state management feeding a formal planner) is not inherently specific to this domain. Several application areas exhibit structural similarities that make the approach transferable with moderate adaptation.

Warehouse and logistics sorting. Automated sorting and goods-to-person fulfilment systems face analogous reconfiguration demands: routing rules, zone assignments, priority designations, and container roles change with incoming orders, product categories, and storage policies. Operators in these environments similarly lack the ability to express such changes directly through existing WMS interfaces without specialist involvement. A conversational layer following the architecture of this thesis, grounded in a live scan of the workspace and maintaining symbolic

routing state across shifts, could reduce the operator effort required to adapt sorting logic without modifying system code.

Laboratory automation. Liquid handling robots, sample processing stations, and automated pipetting platforms in life science laboratories are reconfigured frequently as experimental protocols change. Protocol changes involve the same semantic categories as production changeovers: which containers are sources, which are targets, what volumes and sequences apply, and what priority or fragility constraints govern specific samples. Laboratory staff are domain experts in biology or chemistry, not robotics, making the non-expert accessibility of a conversational interface directly relevant. The PDDL planning component would require domain adaptation but the configuration dialogue architecture transfers directly.

Small and medium enterprise (SME) manufacturing. The industrial motivation established in Chapter 1 applies most acutely to SMEs, which typically lack dedicated robotics engineering staff. In this setting, even the current system’s laboratory-scale capabilities (configuring sorting and kitting tasks through natural language with no code changes) would represent a practical improvement over the manual reconfiguration workflows currently in use. SME deployments would benefit most from the local LLM direction identified in future work, as external API dependency is a significant adoption barrier in environments with limited IT infrastructure.

Agricultural robotics. Harvesting and post-harvest handling systems (picking robots, grading lines, packing stations) involve reconfiguration decisions around crop priority, handling rules for fragile produce, and container assignments that change with harvest conditions and customer orders. An operator directing a picking robot to treat certain crops as fragile, prioritise a specific field zone, or switch between packing configurations is performing the same class of semantic change as the operator in this thesis. The vision component would require adaptation to outdoor conditions and organic shape variation, but the conversational and planning layers are domain-independent at the architecture level.

6.2.2 Uniqueness and position in the literature

As established in Section 2.8, the specific research gap this thesis addresses is the absence of systems that support *iterative, operator-driven maintenance* of a formal world state representation across reconfiguration sessions in modular industrial manipulation. Existing LLM-to-PDDL systems (LLM+P, ViLaIn, Image2PDDL) treat world state generation as a single-shot autonomous inference from a snapshot of perception or text, without operator involvement and without addressing the challenge of preserving semantic attributes that cannot be re-inferred from vision alone. Existing dialogue-based robot control systems address ambiguity resolution and clarification but do not produce formal planning representations or maintain symbolic state across sessions. No prior system integrates all of these capabilities (conversational multi-turn interaction, vision-grounded identity resolution across sessions, and classical PDDL planning) into a single end-to-end operational loop targeted at production reconfiguration.

The thesis therefore occupies a distinct position at the intersection of four re-

search areas: human-centred robot programming, dialogue-based ambiguity resolution, vision-based grounding, and classical task planning. While each of these areas is individually well-developed, their integration in the specific context of iterative operator-driven world state maintenance for industrial manipulation has not previously been addressed. The experimental characterisation of dialogue structure across ambiguity types and task domains additionally provides empirical data on how conversational reconfiguration behaves in practice, data that is largely absent from the existing literature, which tends to evaluate language-robot systems on planning correctness or task success without characterising the interaction structure that produced the outcome.

The scope of the findings is, however, intentionally limited. The results characterise system behaviour in a controlled laboratory setting with a small workspace, a restricted object vocabulary, and a single primary evaluator. They demonstrate feasibility and expose failure modes rather than establishing performance bounds or comparative advantages over alternative approaches. Generalisation to the full diversity of industrial production environments, operator populations, and task domains requires further work along the directions identified in Section 6.3.

6.3 Future work and research

Several directions for future work follow directly from the findings and limitations identified in this thesis.

Broader operator evaluation and comparative baseline. The evaluation was conducted primarily by the system developer, with only two supplementary volunteer trials. While these provide initial evidence that the conversational interface is accessible to naive users, generalisation to a broader operator population is not supported. Future work should include a structured user study with a diverse group of participants (including operators without technical backgrounds) measuring both configuration correctness and perceived interaction effort across varied instruction phrasings. Such a study would also expose phrasing patterns that the current LLM handles inconsistently, providing concrete targets for prompt engineering or fine-tuning. A natural extension of this same study is to establish a comparative dialogue-cost baseline against alternative reconfiguration approaches, such as a purpose-built operator GUI with attribute fields, in order to translate the internal turn-cost characterisation of this thesis into a defensible absolute efficiency judgement. The structural cost drivers identified in Chapter 5 (referential ambiguity overhead, omission cascade in interdependent domains, stateless violation recovery) define what such a comparison would need to measure.

Explicit parameter schema per task mode. The kitting domain revealed that correct configuration outcomes depended on operator intervention because the system does not maintain an explicit checklist of required parameters for each supported task mode. A natural extension is to define, for each operational mode (sorting, kitting, and any future modes), a structured schema specifying which parameters are mandatory, which are optional, and what default values apply when the operator does not specify them. For example, batch size in kitting mode is optional in the sense that the system can produce kits without a fixed batch limit, but this distinc-

tion is currently implicit and leads to ambiguity about whether a missing batch size is an omission to be resolved or a valid underspecification. Making this schema explicit and verifying completeness against it before issuing a proposal would directly prevent the operator-dependent recovery pattern observed in Scenario 3 Experiment 2. This would also enable the system to clearly communicate to the operator which parameters remain unset and whether they are required or optional, reducing the cognitive burden of tracking configuration state across multi-turn sessions.

Local LLM deployment. The current system uses a commercial LLM accessed via API. For industrial adoption, this presents concerns around data security, offline operation, and vendor dependency. A production environment may contain commercially sensitive product configurations, part identities, and batch records that should not be transmitted to external services. Future work should investigate whether locally deployable open-weight models (such as quantised variants of current state-of-the-art models) can replicate the dialogue quality and ambiguity resolution behaviour observed with the current commercial model. If a local model performs comparably, it would substantially lower the barrier to industrial deployment and eliminate the operational risk of API availability or model updates changing system behaviour between sessions.

Stateful constraint violation handling. The Phase 3 loop in Scenario 4 demonstrated that the constraint-violation handler re-evaluates the same state on every turn without retaining memory of prior violation responses. A straightforward improvement is to introduce a violation history within each session: once a violation has been communicated and the operator has responded, the system should either accept the operator’s resolution strategy or reframe the warning rather than re-raising the same message. A more ambitious extension would allow the system to propose concrete resolution strategies (such as suggesting which output roles to remove or which parameters to reset) rather than requiring the operator to determine the restructuring manually.

7. References

- [1] Z. L. Gan, S. N. Musa, and H. J. Yap, “A review of the high-mix, low-volume manufacturing industry,” *Applied Sciences*, vol. 13, no. 3, Jan. 28, 2023, ISSN: 2076-3417. DOI: [10.3390/app13031687](https://doi.org/10.3390/app13031687). Accessed: Jan. 27, 2026. [Online]. Available: <https://www.mdpi.com/2076-3417/13/3/1687>.
- [2] M. Pech and J. Vrchota, “The product customization process in relation to industry 4.0 and digitalization,” *Processes*, vol. 10, no. 3, Mar. 9, 2022, ISSN: 2227-9717. DOI: [10.3390/pr10030539](https://doi.org/10.3390/pr10030539). Accessed: Jan. 31, 2026. [Online]. Available: <https://www.mdpi.com/2227-9717/10/3/539>.
- [3] Y. Dmytriiev, M. Carnevale, and H. Giberti, “Enhancing flexibility and safety: Collaborative robotics for material handling in end-of-line industrial operations,” *Procedia Computer Science*, vol. 232, pp. 2588–2597, 2024, ISSN: 18770509. DOI: [10.1016/j.procs.2024.02.077](https://doi.org/10.1016/j.procs.2024.02.077). Accessed: Jan. 30, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S1877050924002540>.
- [4] “Using robot skills for flexible reprogramming of pick operations in industrial scenarios:” in *Proceedings of the 9th International Conference on Computer Vision Theory and Applications*, Lisbon, Portugal: SCITEPRESS - Science, 2014, pp. 678–685, ISBN: 978-989-758-003-1 978-989-758-004-8 978-989-758-009-3. DOI: [10.5220/0004752306780685](https://doi.org/10.5220/0004752306780685). Accessed: Jan. 30, 2026. [Online]. Available: <http://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0004752306780685>.
- [5] V. Villani, F. Pini, F. Leali, C. Secchi, and C. Fantuzzi, “Survey on human-robot interaction for robot programming in industrial applications,” *IFAC-PapersOnLine*, 16th IFAC Symposium on Information Control Problems in Manufacturing INCOM 2018, vol. 51, no. 11, pp. 66–71, Jan. 1, 2018, ISSN: 2405-8963. DOI: [10.1016/j.ifacol.2018.08.236](https://doi.org/10.1016/j.ifacol.2018.08.236). Accessed: Jan. 27, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405896318313600>.
- [6] G. Ajaykumar, M. Steele, and C.-M. Huang, “A survey on end-user robot programming,” *ACM Comput. Surv.*, vol. 54, no. 8, pp. 164:1–164:36, 2021, ISSN: 0360-0300. DOI: [10.1145/3466819](https://doi.org/10.1145/3466819). Accessed: Jan. 27, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3466819>.
- [7] S. Peldszus, D. Brugali, D. Strüber, P. Pelliccione, and T. Berger, “Software reconfiguration in robotics,” *Empirical Software Engineering*, vol. 30, no. 4, p. 94, Apr. 8, 2025, ISSN: 1573-7616. DOI: [10.1007/s10664-024-10596-9](https://doi.org/10.1007/s10664-024-10596-9). Accessed: Jan. 27, 2026. [Online]. Available: <https://doi.org/10.1007/s10664-024-10596-9>.
- [8] E. Chisari, J. O. v. Hartz, F. Despinoy, and A. Valada, *Robotic task ambiguity resolution via natural language interaction*, Apr. 24, 2025. DOI: [10.48550/arXiv.2504.17748](https://doi.org/10.48550/arXiv.2504.17748). arXiv: [2504.17748\[cs\]](https://arxiv.org/abs/2504.17748). Accessed: Jan. 30, 2026. [Online]. Available: <http://arxiv.org/abs/2504.17748>.

- [9] F. Aghili and K. Parsa, “Configuration control and recalibration of a new reconfigurable robot,” in *Proceedings 2007 IEEE International Conference on Robotics and Automation*, Rome, Italy: IEEE, Apr. 2007, pp. 4077–4083, ISBN: 978-1-4244-0602-9 978-1-4244-0601-2. DOI: [10.1109/ROBOT.2007.364105](https://doi.org/10.1109/ROBOT.2007.364105). Accessed: Feb. 1, 2026. [Online]. Available: <http://ieeexplore.ieee.org/document/4209723/>.
- [10] P. Tsarouchi, S. Makris, and G. Chryssolouris, “Human – robot interaction review and challenges on task planning and programming,” *International Journal of Computer Integrated Manufacturing*, vol. 29, no. 8, pp. 916–931, Aug. 2, 2016, ISSN: 0951-192X, 1362-3052. DOI: [10.1080/0951192X.2015.1130251](https://doi.org/10.1080/0951192X.2015.1130251). Accessed: Feb. 1, 2026. [Online]. Available: <http://www.tandfonline.com/doi/full/10.1080/0951192X.2015.1130251>.
- [11] C. Willibald, L. Martensen, T. Eiband, and D. Lee, *Interactive robot programming for surface finishing via task-centric mixed reality interfaces*, Dec. 29, 2025. DOI: [10.48550/arXiv.2512.23616](https://doi.org/10.48550/arXiv.2512.23616). arXiv: [2512.23616\[cs\]](https://arxiv.org/abs/2512.23616). Accessed: Feb. 1, 2026. [Online]. Available: <http://arxiv.org/abs/2512.23616>.
- [12] R. Ghzouli, T. Berger, E. B. Johnsen, A. Wasowski, and S. Dragule, “Behavior trees and state machines in robotics applications,” *IEEE Transactions on Software Engineering*, vol. 49, no. 9, pp. 4243–4267, Sep. 2023, ISSN: 0098-5589, 1939-3520, 2326-3881. DOI: [10.1109/TSE.2023.3269081](https://doi.org/10.1109/TSE.2023.3269081). arXiv: [2208.04211\[cs\]](https://arxiv.org/abs/2208.04211). Accessed: Feb. 1, 2026. [Online]. Available: <http://arxiv.org/abs/2208.04211>.
- [13] M. Wojtynek and S. Wrede, “InteractiveWorkspace layout focusing on the reconfiguration with collaborative robots in modular production systems,” in *ISR 2020; 52th International Symposium on Robotics*, Dec. 2020, pp. 1–8. Accessed: Feb. 3, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/9307470/>.
- [14] M. Alwi, G. Kajang, and M. Irwanto Mei, “Implementing modular layouts to increase scalability of production facilities,” *Journal of the American Institute*, vol. 2, no. 1, pp. 64–72, 2025. DOI: [10.71364/r93vyj15](https://doi.org/10.71364/r93vyj15). Accessed: Jan. 2, 2026. [Online]. Available: <https://doi.org/10.71364/r93vyj15>.
- [15] H. Owanda, D. W. Utama, and A. Halim, “DESIGN CONTROL SYSTEM OF MODULAR PRODUCTION SYSTEM DISTRIBUTION AND PICK AND PLACE STATION,” *International Journal of Application on Sciences, Technology and Engineering*, vol. 1, no. 3, pp. 912–920, 2023, ISSN: 2987-2499. DOI: [10.24912/ijaste.v1.i3.912-920](https://doi.org/10.24912/ijaste.v1.i3.912-920). Accessed: Feb. 3, 2026. [Online]. Available: <https://journal.untar.ac.id/index.php/ijaste/article/view/31217>.
- [16] J. Seo, J. Paik, and M. Yim, “Modular reconfigurable robotics,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 2, pp. 63–88, Volume 2, 2019 May 3, 2019, ISSN: 2573-5144. DOI: [10.1146/annurev-control-053018-023834](https://doi.org/10.1146/annurev-control-053018-023834). Accessed: Feb. 3, 2026. [Online]. Available: <https://www.annualreviews.org/content/journals/10.1146/annurev-control-053018-023834>.

- [17] W. He, Z. Li, and C. L. P. Chen, “A survey of human-centered intelligent robots: Issues and challenges,” *IEEE/CAA Journal of Automatica Sinica*, vol. 4, no. 4, pp. 602–609, 2017, ISSN: 2329-9274. DOI: [10.1109/JAS.2017.7510604](https://doi.org/10.1109/JAS.2017.7510604). Accessed: Feb. 4, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/8039018/>.
- [18] J. P. C. Verhoosel and M. A. van Bekkum, “Recipe-based engineering and operator support for flexible configuration of high-mix assembly,” in *Advances in Production Management Systems. The Path to Intelligent, Collaborative and Sustainable Manufacturing*, H. Lödding, R. Riedel, K.-D. Thoben, G. von Cieminski, and D. Kiritsis, Eds., Cham: Springer International Publishing, 2017, pp. 363–371, ISBN: 978-3-319-66923-6. DOI: [10.1007/978-3-319-66923-6_43](https://doi.org/10.1007/978-3-319-66923-6_43).
- [19] S. Fixl, M. Hofmann, H. Zörrer, and A. Pichler, “Overcoming challenges in programming human-robot collaboration in flexible manufacturing,” *Procedia Computer Science*, 6th International Conference on Industry 4.0 and Smart Manufacturing, vol. 253, pp. 1740–1749, Jan. 1, 2025, ISSN: 1877-0509. DOI: [10.1016/j.procs.2025.01.236](https://doi.org/10.1016/j.procs.2025.01.236). Accessed: Mar. 18, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050925002443>.
- [20] H. Hu, J. Chen, H. Liu, Z. Li, and L. Huang, “Natural language-based automatic programming for industrial robots,” *Journal of Grid Computing*, vol. 20, no. 3, p. 26, Aug. 3, 2022, ISSN: 1572-9184. DOI: [10.1007/s10723-022-09618-x](https://doi.org/10.1007/s10723-022-09618-x). Accessed: Feb. 5, 2026. [Online]. Available: <https://doi.org/10.1007/s10723-022-09618-x>.
- [21] H. Jones and S. Rock, “Dialogue-based human-robot interaction for space construction teams,” in *Proceedings, IEEE Aerospace Conference*, vol. 7, Mar. 2002, pp. 7–7. DOI: [10.1109/AERO.2002.1035340](https://doi.org/10.1109/AERO.2002.1035340). Accessed: Feb. 5, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/1035340/>.
- [22] M. Stenmark and P. Nugues, “Natural language programming of industrial robots,” in *IEEE ISR 2013*, Seoul, Korea (South): IEEE, Oct. 2013, pp. 1–5, ISBN: 978-1-4799-1173-8. DOI: [10.1109/ISR.2013.6695630](https://doi.org/10.1109/ISR.2013.6695630). Accessed: Feb. 5, 2026. [Online]. Available: <http://ieeexplore.ieee.org/document/6695630/>.
- [23] D. Nyga et al., “Grounding robot plans from natural language instructions with incomplete world knowledge,” in *Proceedings of The 2nd Conference on Robot Learning*, PMLR, Oct. 23, 2018, pp. 714–723. Accessed: Feb. 5, 2026. [Online]. Available: <https://proceedings.mlr.press/v87/nyga18a.html>.
- [24] J. Wang et al., “Large language models for robotics: Opportunities, challenges, and perspectives,” *Journal of Automation and Intelligence*, vol. 4, no. 1, pp. 52–64, Mar. 2025, ISSN: 29498554. DOI: [10.1016/j.jai.2024.12.003](https://doi.org/10.1016/j.jai.2024.12.003). Accessed: Feb. 5, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S2949855424000613>.

- [25] J. Zhong, C. Ling, A. Cangelosi, A. Lotfi, and X. Liu, “On the gap between domestic robotic applications and computational intelligence,” *Electronics*, vol. 10, no. 7, Mar. 27, 2021, ISSN: 2079-9292. DOI: [10.3390/electronics10070793](https://doi.org/10.3390/electronics10070793). Accessed: Feb. 5, 2026. [Online]. Available: <https://www.mdpi.com/2079-9292/10/7/793>.
- [26] P. Pramanick, C. Sarkar, S. Banerjee, and B. Bhowmick, “Talk-to-resolve: Combining scene understanding and spatial dialogue to resolve granular task ambiguity for a collocated robot,” *Robotics and Autonomous Systems*, vol. 155, p. 104 183, Sep. 2022, ISSN: 09218890. DOI: [10.1016/j.robot.2022.104183](https://doi.org/10.1016/j.robot.2022.104183). Accessed: Feb. 5, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0921889022001063>.
- [27] K. Liang, Z. Zhang, and J. F. Fisac, “Introspective planning: Aligning robots’ uncertainty with inherent task ambiguity,”
- [28] T. Degani and N. Tokowicz, “Semantic ambiguity within and across languages: An integrative review,” *Quarterly Journal of Experimental Psychology*, vol. 63, no. 7, pp. 1266–1303, Jul. 1, 2010, ISSN: 1747-0218. DOI: [10.1080/17470210903377372](https://doi.org/10.1080/17470210903377372). Accessed: Feb. 5, 2026. [Online]. Available: <https://doi.org/10.1080/17470210903377372>.
- [29] C. Carpentier and M. Mainguenaud, “Classifying ambiguities in a visual spatial language,” *GeoInformatica*, vol. 6, no. 3, pp. 285–316, Sep. 1, 2002, ISSN: 1573-7624. DOI: [10.1023/A:1019770003896](https://doi.org/10.1023/A:1019770003896). Accessed: Feb. 5, 2026. [Online]. Available: <https://doi.org/10.1023/A:1019770003896>.
- [30] S. E. Duffy and V. Evans, “The top trumps of time: Factors motivating the resolution of temporal ambiguity,” *Language and Cognition*, vol. 9, no. 2, pp. 293–315, Jun. 2017, ISSN: 1866-9808, 1866-9859. DOI: [10.1017/langcog.2016.8](https://doi.org/10.1017/langcog.2016.8). Accessed: Feb. 5, 2026. [Online]. Available: https://www.cambridge.org/core/product/identifier/S1866980816000089/type/journal_article.
- [31] K. D. Dhole, *Resolving intent ambiguities by retrieving discriminative clarifying questions*, Dec. 6, 2024. DOI: [10.48550/arXiv.2008.07559](https://doi.org/10.48550/arXiv.2008.07559). arXiv: [2008.07559\[cs\]](https://arxiv.org/abs/2008.07559). Accessed: Feb. 5, 2026. [Online]. Available: <http://arxiv.org/abs/2008.07559>.
- [32] J. Y. Chai et al., “Collaborative effort towards common ground in situated human-robot dialogue,” in *Proceedings of the 2014 ACM/IEEE international conference on Human-robot interaction*, ser. HRI ’14, New York, NY, USA: Association for Computing Machinery, 2014, pp. 33–40, ISBN: 978-1-4503-2658-2. DOI: [10.1145/2559636.2559677](https://doi.org/10.1145/2559636.2559677). Accessed: Feb. 5, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/2559636.2559677>.
- [33] T. Fong, C. Thorpe, and C. Baur, “Collaboration, dialogue, and human-robot interaction,”
- [34] J. Thomason et al., “Improving grounded natural language understanding through human-robot dialog,” in *2019 International Conference on Robotics and Automation (ICRA)*, ISSN: 2577-087X, May 2019, pp. 6934–6941. DOI: [10.1109/ICRA.2019.8794287](https://doi.org/10.1109/ICRA.2019.8794287). Accessed: Feb. 5, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/8794287/>.

- [35] v.-T. Nguyen, P.-T. Nguyen, S.-F. Su, P. X. Tan, and T.-L. Bui, “Vision-based pick and place control system for industrial robots using an eye-in-hand camera,” *IEEE Access*, vol. 13, pp. 25 127–25 140, 2025, ISSN: 2169-3536. DOI: [10.1109/ACCESS.2025.3536496](https://doi.org/10.1109/ACCESS.2025.3536496). Accessed: Feb. 6, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/10858145/>.
- [36] N. Manakitsa, G. S. Maraslidis, L. Moysis, and G. F. Fragulis, “A review of machine learning and deep learning for object detection, semantic segmentation, and human action recognition in machine and robotic vision,” *Technologies*, vol. 12, no. 2, Jan. 23, 2024, ISSN: 2227-7080. DOI: [10.3390/technologies12020015](https://doi.org/10.3390/technologies12020015). Accessed: Feb. 6, 2026. [Online]. Available: <https://www.mdpi.com/2227-7080/12/2/15>.
- [37] A. Chapin, E. Dellandrea, and L. Chen, *Is an object-centric representation beneficial for robotic manipulation ?* Jun. 24, 2025. DOI: [10.48550/arXiv.2506.19408](https://doi.org/10.48550/arXiv.2506.19408). arXiv: [2506.19408\[cs\]](https://arxiv.org/abs/2506.19408). Accessed: Feb. 6, 2026. [Online]. Available: <http://arxiv.org/abs/2506.19408>.
- [38] Y. Chen and M. Deng, *From perception to symbolic task planning: Vision-language guided human-robot collaborative structured assembly*, Jan. 2, 2026. DOI: [10.48550/arXiv.2601.00978](https://doi.org/10.48550/arXiv.2601.00978). arXiv: [2601.00978\[cs\]](https://arxiv.org/abs/2601.00978). Accessed: Feb. 6, 2026. [Online]. Available: <http://arxiv.org/abs/2601.00978>.
- [39] L. Yang, Y. Xu, C. Yuan, W. Liu, B. Li, and W. Hu, “Improving visual grounding with visual-linguistic verification and iterative reasoning,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, New Orleans, LA, USA: IEEE, Jun. 2022, pp. 9489–9498, ISBN: 978-1-6654-6946-3. DOI: [10.1109/CVPR52688.2022.00928](https://doi.org/10.1109/CVPR52688.2022.00928). Accessed: Feb. 6, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/9879831/>.
- [40] R. Shao et al., *Large VLM-based vision-language-action models for robotic manipulation: A survey*, Sep. 1, 2025. DOI: [10.48550/arXiv.2508.13073](https://doi.org/10.48550/arXiv.2508.13073). arXiv: [2508.13073\[cs\]](https://arxiv.org/abs/2508.13073). Accessed: Feb. 6, 2026. [Online]. Available: <http://arxiv.org/abs/2508.13073>.
- [41] N. Schütte, B. Mac Namee, and J. Kelleher, “Robot perception errors and human resolution strategies in situated human-robot dialogue,” *Advanced Robotics*, vol. 31, no. 5, pp. 243–257, Mar. 4, 2017, ISSN: 0169-1864, 1568-5535. DOI: [10.1080/01691864.2016.1268973](https://doi.org/10.1080/01691864.2016.1268973). Accessed: Feb. 6, 2026. [Online]. Available: <https://www.tandfonline.com/doi/full/10.1080/01691864.2016.1268973>.
- [42] C. Aeronautiques et al., “Pddl—the planning domain definition language,” *Technical Report, Tech. Rep.*, 1998.
- [43] M. Helmert, “The fast downward planning system,” *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, Jul. 12, 2006, ISSN: 1076-9757. DOI: [10.1613/jair.1705](https://doi.org/10.1613/jair.1705). Accessed: Mar. 12, 2026. [Online]. Available: <https://www.jair.org/index.php/jair/article/view/10457>.

- [44] R. Liu, G. Wan, M. Jiang, H. Chen, and P. Zeng, “Autonomous robot task execution in flexible manufacturing: Integrating PDDL and behavior trees in ARIAC 2023,” *Biomimetics*, vol. 9, no. 10, Oct. 10, 2024, ISSN: 2313-7673. DOI: [10.3390/biomimetics9100612](https://doi.org/10.3390/biomimetics9100612). Accessed: Mar. 12, 2026. [Online]. Available: <https://www.mdpi.com/2313-7673/9/10/612>.
- [45] B. Liu et al., *LLM+p: Empowering large language models with optimal planning proficiency*, Sep. 27, 2023. DOI: [10.48550/arXiv.2304.11477](https://doi.org/10.48550/arXiv.2304.11477). arXiv: [2304.11477\[cs\]](https://arxiv.org/abs/2304.11477). Accessed: Mar. 12, 2026. [Online]. Available: <http://arxiv.org/abs/2304.11477>.
- [46] K. Shirai et al., “Vision-language interpreter for robot task planning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, May 2024, pp. 2051–2058. DOI: [10.1109/ICRA57147.2024.10611112](https://doi.org/10.1109/ICRA57147.2024.10611112). Accessed: Mar. 12, 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10611112>.
- [47] X. Dang, L. Kudláčková, and S. Edelkamp, *Planning with vision-language models and a use case in robot-assisted teaching*, Jan. 29, 2025. DOI: [10.48550/arXiv.2501.17665](https://doi.org/10.48550/arXiv.2501.17665). arXiv: [2501.17665\[cs\]](https://arxiv.org/abs/2501.17665). Accessed: Mar. 12, 2026. [Online]. Available: <http://arxiv.org/abs/2501.17665>.
- [48] S. Gabl, *Github repo thesis*, Apr. 27, 2026. Accessed: Apr. 27, 2026. [Online]. Available: <https://github.com/Sandro4433/Thesis>.
- [49] S. Gabl, *Github repo franka demo*, Apr. 27, 2026. Accessed: Jun. 27, 2026. [Online]. Available: https://github.com/Sandro4433/Franka_Emika_Panda_Demo.
- [50] W. Lin, P. Liang, G. Luo, Z. Zhao, and C. Zhang, “Research of online hand–eye calibration method based on ChArUco board,” *Sensors*, vol. 22, no. 10, p. 3805, Jan. 2022, ISSN: 1424-8220. DOI: [10.3390/s22103805](https://doi.org/10.3390/s22103805). Accessed: Feb. 25, 2026. [Online]. Available: <https://www.mdpi.com/1424-8220/22/10/3805>.
- [51] M. Kalaitzakis, B. Cain, S. Carroll, A. Ambrosi, C. Whitehead, and N. Vitzilaios, “Fiducial markers for pose estimation,” *Journal of Intelligent & Robotic Systems*, vol. 101, no. 4, p. 71, Mar. 26, 2021, ISSN: 1573-0409. DOI: [10.1007/s10846-020-01307-9](https://doi.org/10.1007/s10846-020-01307-9). Accessed: Feb. 25, 2026. [Online]. Available: <https://doi.org/10.1007/s10846-020-01307-9>.
- [52] E. Gedraite and M. Hadad, *Investigation on the effect of a Gaussian Blur in image filtering and segmentation*. Jan. 1, 2011, 393 pp., Pages: 396, ISBN: 978-1-61284-949-2.
- [53] R. Sharma and A. Kamra, “A review on CLAHE based enhancement techniques,” vol. 6, Sep. 2023, pp. 321–325. DOI: [10.1109/IC3I59117.2023.10397722](https://doi.org/10.1109/IC3I59117.2023.10397722). Accessed: Apr. 28, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/10397722/>.
- [54] G. Kulkarni Shivananda, “Llms on the fly: Text-to-json for custom api calling,” 2023, pp. 117–154. DOI: [10.1007/978-1-4842-9994-4_7](https://doi.org/10.1007/978-1-4842-9994-4_7). [Online]. Available: https://link.springer.com/10.1007/978-1-4842-9994-4_7.

- [55] J. Liang et al., “Code as policies: Language model programs for embodied control,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, May 2023, pp. 9493–9500. DOI: [10.1109/ICRA48891.2023.10160591](https://doi.org/10.1109/ICRA48891.2023.10160591). Accessed: Mar. 1, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/10160591/>.
- [56] M. Ahn et al., *Do as i can, not as i say: Grounding language in robotic affordances*, Aug. 16, 2022. DOI: [10.48550/arXiv.2204.01691](https://doi.org/10.48550/arXiv.2204.01691). arXiv: [2204.01691\[cs\]](https://arxiv.org/abs/2204.01691). Accessed: Mar. 1, 2026. [Online]. Available: <http://arxiv.org/abs/2204.01691>.
- [57] L. Yuan, D.-J. Han, S. Wang, and C. Brinton, “Local-cloud inference offloading for LLMs in multi-modal, multi-task, multi-dialogue settings,” in *Proceedings of the Twenty-sixth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*, ser. MobiHoc ’25, New York, NY, USA: Association for Computing Machinery, 2025, pp. 201–210, ISBN: 979-8-4007-1353-8. DOI: [10.1145/3704413.3764429](https://doi.org/10.1145/3704413.3764429). Accessed: Mar. 1, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3704413.3764429>.